

MADRE

Agentic
System

• MODEL-AGNOSTIC • DELAYED • REASONING • EFFORT • [Beta v0.1]

Project Definition and Architecture Documentation

Scvria Software

June 23, 2026

Contents

Front Matter	1
Research Foundation	5
2.1 Research Problem	5
2.2 Research Questions	5
2.3 Hypotheses	6
2.4 Expected Contributions	6
2.5 Novelty And Validation Strategy	6
2.6 Related Work And Evidence Map	7
Product Definition	8
3.1 Executive Definition	8
3.2 Product Rationale	8
3.2.1 Extended Rationale	8
3.3 Product Design	10
3.3.1 Product Scope	10
3.3.2 What MADRE Is	10
3.3.3 What MADRE Is Not	10
3.3.4 Core User Value	11
3.3.5 Design Principles	11
3.3.6 Product Boundaries	11
Operational Scenarios	13
4.1 Scenario Cards	13
User Stories And Reference Use Cases	14
5.1 Baseline Purpose	14
5.2 Representative User Stories	14
5.3 Cross-Use-Case Conclusions	17
Conceptual Architecture	20
6.1 Formal Requirements	20
6.2 High-Level Architecture	20
6.2.1 Conceptual Components	21
6.2.2 Responsibility Layer Diagram	22
6.2.3 Concurrent Agent Lanes	22
6.3 Runtime Data Flow	22
6.3.1 Canonical Request Narrative	23

6.3.2	Governed Model And Action Sequence	23
6.3.3	Data And Authority Rules	26
6.3.4	Trust Boundary Diagram	27
6.3.5	Access Surfaces	27
Safety And Evolution Policy		29
7.1	Runtime Safety	29
7.1.1	Policy Before Action	29
7.1.2	Autonomy Levels	29
7.2	Evolution Control	30
7.2.1	Evolution Rules	30
System Engineering Baseline		31
8.1	Engineering Problem	31
8.2	Stakeholders And Concerns	31
8.3	Requirement Domains	31
8.4	Traceability Surface	32
8.5	Quality Attributes	32
8.6	Risk And Policy Boundaries	32
Requirements Baseline		33
9.1	Requirement Taxonomy	33
9.2	Functional Requirements	34
9.3	Cross-Cutting Requirements	35
Traceability Model		38
10.1	Traceability Method	38
10.2	Architecture Traceability Matrix	38
10.3	Integrated Architecture Trace	39
Design Decision Traceability		41
Architecture Viewpoints		43
12.1	Viewpoint Catalogue	43
12.2	Viewpoint Design Focus	44
Runtime Governance		46
13.1	Runtime Authority Rule	46
13.2	Policy And Action Gate	46
13.3	Policy Matrix	46
13.4	Learning Lifecycle	47
Threat Model		48
14.1	Threat Taxonomy	48
Quality Model		50
15.1	Quality Attribute Scenarios	50

UX, Observability, And Recovery	52
16.1 Interaction Contract	52
16.2 Observability Surfaces	52
Evaluation And Benchmark Specification	53
17.1 Benchmark Families	53
Governance, Deployment, And Limits	55
18.1 Secure Engineering Principles	55
18.2 Open-Source Governance	55
18.3 Deployment And Operational Assumptions	55
18.4 Risks, Limitations, And Excluded Functions	55
Runtime Domain Model	56
19.1 Design Rule	56
19.2 Minimal Canonical Flow	56
19.3 Conventions	57
19.3.1 Type Conventions	57
19.3.2 Process Conventions	57
19.3.3 Class Inventory	58
19.4 Accepted UML Registry	59
19.5 Structural Relations	59
19.5.1 Runtime Ownership	59
19.5.2 Agent Family	60
19.5.3 Planning And Invocation	61
19.5.4 Context, Material, Knowledge, And Learning	62
19.5.5 Trace And Authority Evidence	62
19.6 Class Definitions	63
19.6.1 MADREKernel	63
19.6.2 ReasoningModule	64
19.6.3 MADREAgent	64
19.6.4 SimpleAgent	65
19.6.5 TwinAgent	65
19.6.6 ComplexAgent	65
19.6.7 SystemAgent	66
19.6.8 Workflow	66
19.6.9 AgentRoutine	66
19.6.10 AgentAction	67
19.6.11 ModelAction	67
19.6.12 ModelBinding	68
19.6.13 ReasoningRequest	68
19.6.14 ReasoningResponse	69
19.6.15 ReasoningPlan	69
19.6.16 ReasoningTask	70
19.6.17 AgentRequest	70
19.6.18 AgentResponse	71
19.6.19 ContextBundle	71
19.6.20 ReasoningArtifact	72

19.6.21 KnowledgeRecord	72
19.6.22 KnowledgeCandidate	73
19.6.23 LearningCandidate	73
19.6.24 PolicyDecision	74
19.6.25 RuntimeJournal	74
19.6.26 RuntimeEvent	75
19.7 Validation Checklist	75

Glossary	77
-----------------	-----------

Front Matter

Abstract

MADRE is a local-first, model-agnostic runtime kernel for building agentic systems that preserve user sovereignty while coordinating reasoning modules, module-owned agents, governed context, bounded actions, known material, evaluated learning, recovery, and inspectable execution.

//

MADRE Agentic System[Beta v0.1]

MADRE exists because the current AI market is normalizing a dangerous bargain: users receive convenient language interfaces, while their data, dependency chain, costs, and decision power move into remote platforms they do not control. That bargain is technically unnecessary and politically unacceptable for individuals, teams, public-interest organizations, researchers, creators, and small developers.

The project does not treat a language model as a complete intelligence. A model is a powerful linguistic function: useful, fallible, statistically trained, expensive in context, and unsafe when it is allowed to become the whole system. **MADRE** puts that function inside an engineered runtime. The runtime decides what may be known, what may be sent, what may be stored, what may be acted on, and what must wait for stronger evidence or human authority.

The product objective is one coherent assistant experience backed by a durable kernel. Visible interaction can remain fast and conversational while module-local reasoning and reflection lanes decompose, schedule, check, record, and resume deeper work. This is the practical meaning of Model Agnostic Delayed Reasoning Effort: useful local intelligence should come from architecture, context discipline, and bounded autonomy, not from blind dependence on one remote provider or one large model.

MADRE is therefore not a chatbot, not a prompt collection, not a wrapper around one vendor API, and not a task-management application. It is the runtime layer that applications use to coordinate models, internal actions, conversation and knowledge records, policies, and interfaces without surrendering the user's private domain to cloud-first AI infrastructure.

Specification Scope

MADRE is a local-first, model-agnostic runtime kernel for constructing agentic systems whose useful behavior is produced by software architecture rather than by treating language models as autonomous intelligence. The runtime assigns a Core Module, permits its Main Agent role to provide foreground continuity through a module-owned **SystemAgent**, governs context before **ModelAction** or other actions, keeps **KnowledgeRecord** handling inspectable, records local traces through **RuntimeJournal**, and requires learning to pass through evaluation, promotion, and rollback boundaries.

This specification defines the research, product, and architecture baseline for **MADRE**. It states the system problem, expected contribution, stakeholder concerns, requirement domains, architecture views,

authority model, and evaluation surface used to assess runtime behavior. Optional remote services and integrations are governed risk boundaries, not architectural defaults.

The scope of **MADRE** comprises runtime purpose, system boundary, requirement families, architecture obligations, risk model, quality attributes, validation strategy, and product-level authority semantics.

The design identifiers D-001 through D-013 are internal traceability anchors inside this dossier. They identify product rationale, runtime obligations, safety boundaries, and evaluation surfaces at architecture level without making prior side specifications authoritative.

Canonical Terms

Term	Meaning in MADRE
MADREKernel	Deterministic local coordinator that registers ReasoningModule objects, assigns the Core Module, routes requests, schedules work, coordinates resources, applies policy, and records runtime activity through RuntimeJournal .
ReasoningModule	Governed reasoning unit for a bounded domain or scope. It owns module records, workflows, routines, actions, agents, knowledge and learning boundaries, and allowed model/inference scope.
Core Module	Kernel registration role held by a compatible ReasoningModule that receives ordinary interaction first; it is not a separate module type.
MADREAgent / Main Agent role	A MADREAgent is a module-owned runtime actor. Main Agent role is held by one module-owned SystemAgent ; other agent profiles do not gain that role by default.
SimpleAgent, TwinAgent, ComplexAgent	Agent profiles for direct execution, review/evaluation over a produced result, and foreground/reasoning/reflection capability respectively; none gains authority outside its owning module.
ReasoningPlan, Workflow, AgentRoutine	A request-derived reasoning plan may select reusable module workflow and agent routine capabilities without turning either into a second plan or a new authority layer.

Term	Meaning in MADRE
Access surface	A CLI, local console, GUI, speech surface, or application interface that exposes the kernel without becoming runtime authority.
ReasoningTask, RuntimeEvent, RuntimeJournal	ReasoningTask is the plan-level unit for delayed work; RuntimeEvent is append-only runtime history; RuntimeJournal is the local trace mechanism that stores or references events and related payloads.
PolicyDecision	Explicit allow, block, or authorization-required outcome for an affected action or data class and authority boundary.
Foreground lane	A possible ComplexAgent capability used by a module's Main Agent role to preserve flow, clarify intent, answer within safe context, and create delayed work when stronger evidence is required.
Reasoning lane	Durable module-local work used for analysis, evidence gathering, verification, internal action invocation, or higher-effort inference.
source material / ContextBundle	Source descriptor and bounded, classified, minimized, revocable selection of material prepared for an agent request, action, or inference execution.
AgentAction, ModelAction, ModelBinding, model-use profile, model backend	An executable capability and its inference-backed specialization; a model-backed action executes only through the binding allowed for its module-owned agent, with configured and replaceable inference execution.
RuntimeEvent evidence / generated material	One recorded inference execution and its generated material. Generated material is not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself.

Term	Meaning in MADRE
KnowledgeRecord / KnowledgeCandidate	Known material held by a module with provenance, scope, sensitivity, intended use, truth-level/authority, correction, and lifecycle metadata; only a pending knowledge-boundary change is a KnowledgeCandidate.
LearningCandidate / retained interaction material	An evaluated proposed module improvement and retained interaction material respectively. Saving known material is not learning, and conversation is not knowledge by default.
Artifact	A materialized output such as a file, document, code patch, report, image, dataset, PDF, snapshot, or final buffer/document. Ordinary conversation text is not an artifact unless materialized.

Specification Baseline

Version	Status
Beta v0.1	Research, product, and architecture specification baseline for local-first, model-agnostic delayed reasoning. The baseline covers product requirements, architecture obligations, safety boundaries, traceability, and evaluation criteria.

Research Foundation

2.1 Research Problem

Modern language-model systems often combine private data, model calls, action execution, retained interaction, known material, and remote infrastructure inside opaque conversational interfaces. This makes useful demonstrations easy to produce, but it weakens local ownership, traceability, autonomy control, error recovery, and the ability to replace models or providers.

MADRE addresses this as a Software Engineering problem: how to design a local-first runtime in which language models remain fallible replaceable functions, while the software system owns policy, context, knowledge, internal actions, runtime trace, recovery, and learning boundaries. The research problem is not to prove that models reason, but to define an architecture that extracts useful language behavior without surrendering system authority to generated material.

2.2 Research Questions

ID	Research question
RQ-1	How can a local-first runtime separate immediate interaction from delayed reasoning without losing continuity, accountability, or user control?
RQ-2	How can model-agnostic contracts prevent provider-specific assumptions from entering policy, knowledge handling, action authority, and truth management?
RQ-3	What traceability structure is sufficient to connect MADRE ’s product claims, stakeholder concerns, runtime requirements, architecture decisions, and validation evidence?
RQ-4	How can context and retained material remain useful while preserving provenance, truth authority, minimization, revocation, and correction boundaries?
RQ-5	What integrated behavior demonstrates that MADRE is a governed runtime rather than a conversational wrapper?

2.3 Hypotheses

ID	Hypothesis
H-1	Module-local foreground, reasoning, and reflection lanes can preserve conversational responsiveness while improving reliability for tasks that require evidence, verification, action execution, or stronger context.
H-2	Model-agnostic contracts can reduce provider lock-in when policy, knowledge handling, context, and action execution remain owned by the runtime.
H-3	Context governance with provenance, classification, minimization, and revocation can reduce leakage and misuse without eliminating useful specialized reasoning.
H-4	Evaluated LearningCandidate promotion can improve system adaptation while preventing generated material or ordinary KnowledgeRecord storage from silently rewriting protected behavior or factual authority.
H-5	RuntimeJournal , ReasoningTask , and append-only RuntimeEvent history can make delayed reasoning inspectable and recoverable enough for ordinary local operation.

2.4 Expected Contributions

MADRE’s expected contribution is a reference architecture for local-first, model-agnostic delayed reasoning. The contribution combines module-owned agents, optional Complex Agent lane profiles, governed context bundles, explicit **ModelAction** and internal action contracts, **RuntimeEvent** evidence/generated material separation, controlled learning, and traceable recovery paths.

The novelty is not a new language model. It is the engineering arrangement that keeps language models useful while preventing them from becoming sources of truth, authority, policy, knowledge, or action. The architecture is intended to be useful for personal assistants, developer systems, organizational knowledge workers, domain systems, and local interfaces that require control over private knowledge.

2.5 Novelty And Validation Strategy

The first research claim is that a useful agentic system can be validated as a software architecture before it is validated as a broad automation product. **MADRE**’s novelty is the conjunction of local-first execution, replaceable inference and action boundaries, delayed reasoning, governed context, explicit **PolicyDecision** outcomes, evaluated learning promotion, and journaled recovery.

The validation strategy examines whether a local request can enter through bounded access surfaces, be triaged without unsupported claims, become a durable **ReasoningTask** when stronger evidence is needed, obtain a governed context bundle, cross a replaceable model backend, invoke a bounded internal action after a permitting **PolicyDecision**, produce generated material that remains non-authoritative, expose **RuntimeJournal** trace evidence, and support correction, invalidation, supersession, rollback, or deletion/detachment.

The validation strategy treats negative paths as first-class evidence. Prompt or source injection must remain source data. Known material used with an incorrect **truthLevel** or **truthAuthority** must be rejected or corrected. Sensitive or unrelated material must be excluded from context. Generated

material must not become truth, policy, action authority, memory, knowledge, behavior, routing, or a candidate by itself. A **KnowledgeCandidate** changes a knowledge boundary only through promotion, and a **LearningCandidate** changes module behavior only after defined evaluation and promotion.

2.6 Related Work And Evidence Map

Evidence family	Relevance to MADRE	Specification use
Local-first software	Establishes user ownership, offline usefulness, privacy, longevity, and optional synchronization as product concerns.	Supports data sovereignty and local-first architecture claims.
Requirements engineering	Provides structure for measurable requirements, acceptance criteria, verification methods, and source traceability.	Supports the SRS and traceability model.
Architecture description	Provides stakeholder, concern, viewpoint, view, correspondence, and decision-rationale concepts.	Supports multi-view architecture description.
LLM and agent security	Identifies prompt injection, sensitive disclosure, supply-chain risk, data poisoning, excessive agency, vector weakness, misinformation, and resource consumption.	Supports threat and policy modeling.
Software quality models	Provide vocabulary for reliability, maintainability, portability, security, performance efficiency, usability, and quality-in-use evaluation.	Supports the quality attribute model.
AI evaluation research	Provides multi-metric and evidence-oriented evaluation patterns for generated material, retrieval quality, and agentic task performance.	Supports MADRE-native benchmark families.

Product Definition

3.1 Executive Definition

MADRE is a runtime foundation rather than a single assistant interface. The kernel assigns ordinary interaction to a Core Module, records delayed lifecycle and runtime history locally, and keeps policy, knowledge, action, and recovery authority outside inference. Language models run through module-constrained **ModelAction** execution; generated material is recorded through **RuntimeEvent** evidence and does not become authority or a candidate by itself.

3.2 Product Rationale

MADRE Agentic System

Rationale	Product implication
Local control	Private data, local knowledge, and sensitive work remain under local authority unless a policy-governed transfer is explicitly permitted.
Model substitutability	No model provider, host, prompt convention, or output format defines runtime truth, policy, knowledge, or action authority.
Delayed reasoning	Work that requires evidence, verification, resource control, or stronger context is represented as a traceable ReasoningTask rather than as an unsupported foreground answer.
Context governance	Context is selected, minimized, classified, sourced, scoped, provenanced, and revocable before it is provided to a ModelAction , internal action, or reasoning task.
Bounded action use	Internal actions, external actions, and knowledge-boundary changes are allowed only through declared authority boundaries, PolicyDecision outcomes, trace evidence, and recovery expectations. Workflows remain planning structure.
Known material and learning	A KnowledgeRecord is not factual truth by default; a KnowledgeCandidate is reserved for pending knowledge-boundary change, while a LearningCandidate is an evaluated proposed module improvement.

3.2.1 Extended Rationale

- **Data Sovereignty:**

The first design obligation of **MADRE** is to keep the user's data under the user's control. Contemporary AI services often make private life, professional work, organizational knowledge,

and creative material flow through remote systems by default. That default is not neutral. It concentrates power in the companies that own the infrastructure, the logs, the model services, the policy switches, and the billing relation.

MADRE refuses to make surveillance a prerequisite for assistance. Local-first operation is a security boundary, a political boundary, and an engineering discipline. Private conversation, module-owned known material, organizational records, and sensitive work must remain local unless an explicit policy, classification, minimization, and trace path says otherwise.

Internet access is treated as a risk boundary. Prompt injection, credential exposure, external action abuse, provider lock-in, and remote data retention are normal failure modes of cloud-first agentic systems; **MADRE** 's architecture must make those failures harder by default and visible when they cannot be eliminated.

- **Free Software Duty:**

MADRE is released under GPL-3.0 because source availability alone is not enough. The project aims to provide usable agentic infrastructure for people and organizations that cannot, or should not, depend on platform owners for every intelligent interface they build.

The current generation of AI models draws on a broad inheritance of human language, labor, research, art, software, and social knowledge. A public-interest response requires practical, inspectable infrastructure outside exclusive corporate channels.

MADRE supports developers and creators running useful systems on ordinary machines, inspecting decisions, replacing inference backends and authorized integrations, and sharing improvements without requiring platform permission.

- **Model Agnosticism:**

No single model, host, vendor, format, or prompt convention should define **MADRE** 's architecture. Models improve, fail, disappear, become expensive, change license terms, and carry different bias and safety profiles.

MADRE treats inference as a governed **ModelAction**. The runtime may choose different permitted model-use profile and model backend combinations without letting any model become authority over policy, knowledge, actions, or truth.

- **Resource Discipline:**

Local-first AI must remain useful on ordinary commercial devices and degrade intelligently when resources are scarce.

Optional Complex Agent lanes, task decomposition, caching, context minimization, repeated checks where they matter, and durable records make stronger reasoning selective rather than an unlimited default.

- **Knowledge Governance:**

Agentic systems become dangerous when they blur text, known material, retained interaction, instruction, evidence, and action. A conversation turn is not a verified fact, generated material is not a source or authority, and an action result is not an instruction.

A **KnowledgeRecord** carries provenance, sensitivity, intended use, **truthLevel**, and **truthAuthority**; a **LearningCandidate** is an evaluated module-improvement proposal, not ordinary stored knowledge.

- **Distributed Reasoning:**

Interaction can remain coherent while module-owned agents defer hard work, retrieve permitted material, test hypotheses, revise plans, and learn from evaluated outcomes without treating language models as minds.

Every agent, `RuntimeEvent` evidence, action invocation, `KnowledgeRecord` change, and `LearningCandidate` must remain accountable to an explicit runtime boundary.

3.3 Product Design

3.3.1 Product Scope

MADRE provides a disciplined coordination layer underneath agentic applications. The kernel routes interaction, registers the Core Module, records delayed work through `RuntimeJournal`, coordinates bounded context and actions, records `PolicyDecision` outcomes, and exposes recovery semantics.

The primary audience is developers and organizations building private, domain-aware, extensible assistant systems. Applications built on **MADRE** may vary in interface, domain, and policy, but they inherit the same authority model: modules own their agents and known material, model execution is constrained by `ModelBinding`, and policy, trace, action, and recovery boundaries remain runtime-controlled.

3.3.2 What MADRE Is

- A local-first runtime kernel for agentic systems.
- A model-agnostic coordination layer for multiple model families.
- A delayed-reasoning architecture that keeps conversation responsive while deeper work happens through module-local agents and `ReasoningTask` lifecycle.
- A policy-aware boundary for context, internal actions, knowledge, learning, and remote transfer.
- A developer foundation for building assistants, domain systems, and workflow-capable applications on top of governed local intelligence.

3.3.3 What MADRE Is Not

- **MADRE** is not a chatbot product. Chat is one possible surface over the kernel, not the kernel's definition.
- **MADRE** is not a prompt wrapper. Prompts are runtime inputs, not an architecture.
- **MADRE** is not a vendor abstraction layer whose purpose is merely to swap API providers.
- **MADRE** is not an unrestricted automation agent. Internal actions, external actions, knowledge-boundary change, and learning require explicit boundaries.
- **MADRE** is not a passive record of interaction history. The product is the governed runtime kernel and its architectural obligations.

3.3.4 Core User Value

MADRE lets an application answer quickly without pretending every answer can be safe, complete, or well-reasoned in the foreground. The Core Module can assign a Main Agent role with a Complex Agent profile whose foreground lane clarifies intent, uses appropriately classified material, names uncertainty, and triggers deeper work. Module-local reasoning lanes perform work requiring decomposition, evidence, internal actions, verification, or stronger context.

This separation gives users a better contract. They are not forced to choose between a quick hallucination and a slow black box. They get immediate conversation when that is appropriate, delayed reasoning when that is necessary, and inspectable boundaries when the system touches sensitive data or meaningful actions.

3.3.5 Design Principles

- **Local-first, not local-only:** Local execution is the default because privacy, cost, resilience, and agency require a strong local base. Remote services may exist only as explicitly governed integrations with classification, minimization, policy, provenance, and journaled trace.
- **Model-agnostic by contract:** Models are replaceable runtime functions. The kernel must not let any model provider define the system's truth model, policy boundary, knowledge representation, or action authority.
- **Foreground lane, durable reasoning:** A Main Agent role may use a Complex Agent foreground lane to preserve conversational flow. Expensive, uncertain, risky, or evidence-heavy work should become a scheduled `ReasoningTask` that can be resumed, inspected, verified, and delivered asynchronously.
- **Context is a governed asset:** Context is not just text stuffed into a prompt. It is selected, minimized, classified, sourced, scoped, and revoked according to the purpose of the task and the authority of the actor using it.
- **Autonomy is bounded:** **MADRE** must support unattended delayed work, but autonomy is legitimate only when the action has an applicable `PolicyDecision`, is recorded through `RuntimeEvent`, is recoverable for its risk level, and is blocked from crossing stronger boundaries without authorization.
- **Learning is evaluated:** Useful systems should improve from use, but a `LearningCandidate` cannot quietly rewrite protected behavior. Saving a `KnowledgeRecord` is not learning; knowledge-boundary and behavior changes must be reviewable, promotable, reversible, and separable.
- **Interfaces do not own the architecture:** The `madre` CLI, local read-first inspection console, graphical UI, speech surfaces, or domain applications are access surfaces. They must expose and operate the kernel without becoming the source of runtime authority.

3.3.6 Product Boundaries

The product boundary covers runtime responsibilities and architecture obligations: local-first operation, model-agnostic inference, module-owned agent coordination, governed context, bounded internal actions, knowledge handling, evaluated learning, runtime traceability, and recovery.

MADRE may expose many application forms: personal assistants, research systems, developer environments, organizational knowledge workers, or domain-specific automation environments. Those applications may differ in interface and policy, but they must inherit the same kernel commitments: local-first operation, replaceable inference, governed context, bounded internal actions, inspectable retained interaction material and **KnowledgeRecord** handling, evaluated learning, journaled runtime history, and recovery.

Operational Scenarios

4.1 Scenario Cards

Scenario trigger	Module(s)	Runtime objects	Authority boundary	Expected records	Negative path
Foreground continuity / ordinary interaction	Core Module	Main Agent role, ContextBundle , retained interaction material	Answer only from permitted context and without making known material factual by implication.	Request and response RuntimeEvent ; retained interaction material where configured.	Ambiguity, insufficient context, or risk creates clarification, block, or ReasoningTask , not an unsupported answer.
Delayed research evidence required	Core and research module	ReasoningPlan , ReasoningTask , ContextBundle , optional ModelAction	Routing, data scope, and each action require applicable PolicyDecision .	Work lifecycle and evidence references in RuntimeJournal ; RuntimeEvent evidence when inference runs.	Unverified or disallowed sources do not become accepted knowledge or authoritative response.
Knowledge handling / material selected for module use	Owning reasoning module	KnowledgeRecord ; KnowledgeCandidate only for pending boundary change	Provenance, scope, sensitivity, intended use, truthLevel , truthAuthority , correction, and lifecycle metadata govern use.	Knowledge creation/correction/detachment RuntimeEvent and related PolicyDecision .	Known material with unsuitable truth metadata is not presented as factual authority; saving it is not learning.
Specialized interaction / domain assistance requested	Core and selected domain module	retained interaction material, ContextBundle , module-owned MADREAgent , ModelBinding	A transferred or delegated scope does not transfer policy or knowledge authority.	Routing event, selected module, binding reference, context reference, and policy outcome.	Domain-specific output remains generated material until explicitly handled; it does not self-route or self-promote.
Workflow-guided execution / reusable behavior invoked	Owning reasoning module	Workflow , AgentRoutine , AgentAction , ReasoningTask	Workflow helps plan work; each executable action and external crossing is policy-gated with recovery expectation.	Action, work, policy, failure, and recovery RuntimeEvent history.	A hidden prompt chain or unauthorized side effect is blocked or authorization-required.
Evaluated improvement / repeated outcome suggests change	Owning reasoning module	LearningCandidate , evaluation evidence, optional KnowledgeRecord input	Promotion changes module behavior only after declared evaluation and promotion authority.	Candidate creation, evaluation, promotion/rejection, and rollback events.	Generated material, conversation retention, or KnowledgeRecord storage alone never activates learning.

Developers may add models, internal actions, authorized external actions, projections, context policies, workflows, interfaces, and analysis helpers without rewriting the kernel, provided module ownership, **ModelBinding**, **PolicyDecision**, and **RuntimeJournal** trace obligations remain intact.

User Stories And Reference Use Cases

5.1 Baseline Purpose

The reference user stories preserve behavior pressure for the runtime class map. They are not implementation scripts. They show where module ownership, context minimization, delayed reasoning, workflow application, agent-routine execution, model-backed actions, knowledge handling, learning proposals, and trace evidence must remain distinguishable.

5.2 Representative User Stories

ID: UC-03

Academic section with recent sources

User Journey student

Epic: Research-grounded academic writing

User Voice: As a **student** I want to write an academic section using recent evidence without leaking unrelated private context

Acceptance:

- Delegates the academic task from **CoreModule** to **CollegeTasksModule**.
- Sends only a minimized, neutral research objective to **ResearchModule**.
- Checks retrieved material for source quality and injection risk.
- Returns evidence-backed draft material through the owning academic module.

ID: UC-04

Private-context leakage prevention

User Journey privacy-conscious user

Epic: Boundary-safe web research

User Voice: As a **privacy-conscious user** I want to search externally without exposing sensitive personal details

Acceptance:

- Detects private context mixed into a web-enabled research request.

- Separates the academic objective from sensitive user details.
- Records a boundary or policy outcome before delegation.
- Gives **ResearchModule** only a safe minimized query, or asks for confirmation.

ID: UC-05

Assignment prompt injection

User Journey student

Epic: Untrusted input handling

User Voice: As a **student** I want to use assignment text as source material without letting hostile instructions execute

Acceptance:

- Reviews pasted assignment text before treating it as task context.
- Classifies malicious instructions as source content, not authority.
- Blocks the harmful instruction from action, policy, or boundary influence.
- Records the security outcome while preserving usable academic content.

ID: UC-08

LaTeX document compile recovery

User Journey student

Epic: Artifact production and recovery

User Voice: As a **student** I want to create a LaTeX assignment document and recover from compilation errors

Acceptance:

- Applies a module workflow when the LaTeX assignment pattern is recognized.
- Invokes the LaTeX writing agent through concrete task execution.
- Materializes source, logs, and outputs as **ReasoningArtifact** records.
- Keeps compile/check actions bounded and records repair attempts.

ID: UC-12

Generated draft becomes retained knowledge

User Journey student

Epic: Knowledge lifecycle

User Voice: As a **student** I want *to remember a useful generated explanation for future assignments*

Acceptance:

- Treats the generated draft first as produced material, not active knowledge.
- Promotes or proposes it as a `KnowledgeRecord` only after explicit handling.
- Scopes the retained material as explanation style or accepted study explanation.
- Records the promotion, reclassification, or capture event.

ID: UC-13

Repeated corrections produce a learning candidate

User Journey user

Epic: Module-local learning

User Voice: As a **user** I want *to have repeated feedback improve future academic writing behavior*

Acceptance:

- Captures correction history as runtime evidence.
- Allows individual corrections to update relevant `KnowledgeRecord` entries.
- Forms a separate `LearningCandidate` for broader behavior change.
- Requires evaluation or promotion before the behavior change becomes active.

ID: UC-18

Queued report under low resources

User Journey user

Epic: Delayed work and inspectability

User Voice: As a **user** I want *to queue a large report when foreground completion is not practical*

Acceptance:

- Creates a `ReasoningPlan` with inspectable `ReasoningTask` objects.

- Responds immediately with queued-work status.
- Schedules execution as resources become available.
- Preserves lifecycle events and final artifacts for recovery and inspection.

ID: UC-20

Background reasoning finds a better answer

User Journey user

Epic: Background supersession

User Voice: As a **user** I want *to receive a corrected or superseding result without losing traceability*

Acceptance:

- Keeps the foreground answer separate from delayed reasoning evidence.
- Records the background result as a new runtime transition.
- Marks the earlier answer as superseded rather than silently replacing it.
- Preserves the reason, source, and artifact links for inspection.

ID: UC-26

LaTeX worksheet with math and research

User Journey student

Epic: Cross-module dependency chain

User Voice: As a **student** I want *to produce a worksheet that combines solved examples, recent references, and LaTeX output*

Acceptance:

- Lets `CollegeTasksModule` own the final academic deliverable.
- Delegates focused reasoning to `MathModule` and minimized research to `ResearchModule`.
- Integrates math results, research notes, and LaTeX artifacts into one plan.
- Records dependency links, validation steps, and final artifact delivery.

5.3 Cross-Use-Case Conclusions

- **KnowledgeRecord:**

A `KnowledgeRecord` is broad retained module material. It may represent fact

preference
style
procedure
fiction
hypothesis
correction
imported material
or generated summary under provenance
scope
sensitivity
intended use
truth authority
confidence
and lifecycle metadata.

- **ReasoningArtifact:**

A **ReasoningArtifact** is produced or materialized output. It may later inform knowledge handling but it is not active knowledge by default.

- **Generated material:**

Generated material is ordinary risky material. It may be useful evidence or output but it cannot create authority

policy
routing
knowledge
action permission
or learning by itself.

- **Boundary composition:**

Executable validity comes from composed constraints across modules

agents
workflows
routines
actions
context
model use
storage access

and learning authority. This must be enforced by construction rather than hidden behind a loose policy flag.

- **Kernel routing:**

The kernel resolves explicit valid targets or falls back to the Core Module. Semantic delegation remains module reasoning.

- **SystemAgent and TwinAgent:**

SystemAgent is justified only as a strictly module-bound **ComplexAgent**. **TwinAgent** is justified by the checked-output invariant not by a particular model or thread layout.

- **Workflow and AgentRoutine:**

Workflow is module-level planning structure that generates or refines **ReasoningTask** objects. **AgentRoutine** is agent-level executable compound capability that receives **AgentRequest** and returns **AgentResponse**.

- **Learning:**

LearningCandidate proposes behavior improvement and remains inactive until evaluated promotion. Learning cannot widen privacy or action boundaries merely because it improves output quality.

Conceptual Architecture

6.1 Formal Requirements

This chapter states architectural requirements at product level.

Requirement	Architectural meaning
Local-first operation	The kernel must be able to run useful workflows on local user-owned infrastructure before optional remote integrations are considered.
Model agnosticism	All model execution enters through <code>ModelAction</code> , allowed <code>ModelBinding</code> , replaceable model-use profile, and model backend contracts, never through a vendor-specific core design.
Delayed reasoning	The runtime must separate immediate conversation from deeper work that needs decomposition, evidence, verification, internal actions, or more compute.
Context governance	Context must be selected, minimized, classified, provenanced, scoped, and revocable before it reaches models or internal actions.
Bounded internal actions	Executable behavior must be declared, policy-checked, traceable, and constrained by side effect and recovery rules. Typed runtime-defined internal actions are the canonical low-risk action form.
Record/knowledge isolation	retained interaction material, <code>KnowledgeRecord</code> , untrusted source material, <code>KnowledgeCandidate</code> , and <code>LearningCandidate</code> must not collapse into one undifferentiated prompt store.
Learning control	A <code>LearningCandidate</code> is an evaluated proposed improvement and changes active module behavior only through a defined promotion boundary; storing a <code>KnowledgeRecord</code> is not learning.
Inspectable execution	The CLI, local developer console, <code>RuntimeJournal</code> , and read models must make runtime behavior understandable without bypassing policy.

6.2 High-Level Architecture

MADRE is organized around a deterministic runtime kernel that coordinates governed reasoning modules through responsibility boundaries rather than through a fixed topology.

The central architectural rule is simple: the model is never the system. Models are invoked through `ModelAction` under context, policy, action, knowledge, and trace constraints. Each concrete **MADREAgent**

is owned by one **ReasoningModule** and constrained by **ModelBinding**. Agents may be short-lived, persistent, specialized, conservative, speculative, or workflow-bound, but they do not own the kernel or carry module knowledge across ownership boundaries.

The kernel coordinates modules; it does not become the reasoning actor for every task. System-level reasoning belongs in governed modules. Kernel registration assigns one compatible **ReasoningModule** the Core Module role for incoming interaction, continuity, clarification, planning, delegation, and delayed-work decisions. Within that module, one module-owned **SystemAgent** holds the Main Agent role; other **ComplexAgent** objects may exist without holding that role.

6.2.1 Conceptual Components

Component	Responsibility
Access surfaces	Present the system through CLI, local read-first inspection console, graphical UI, speech, or domain applications without becoming the source of runtime authority.
Core Module	A kernel-assigned ReasoningModule role that maintains interaction continuity, clarifies intent, creates reasoning plans, delegates to modules, and decides when deeper work is needed.
Runtime kernel	Coordinates module registration, routing, scheduling, resource limits, policies, action registries, durable records, recovery, and lifecycle.
Reasoning modules	Execute bounded reasoning domains through module-owned agents, workflows, routines, actions, records, policies, known-material boundaries, and allowed model/inference scope.
Context and knowledge	Provides scoped context bundles and KnowledgeRecord objects with provenance, scope, sensitivity, intended use, truth metadata, and correction paths.
Local Model Runtime Boundary	Product-level boundary for local or explicitly authorized inference through ModelBinding , model-use profile, model backend, RuntimeEvent evidence, and generated material.
Agent Action boundary	Exposes typed, bounded internal actions with declared inputs, outputs, side-effect class, policy requirements, trace evidence, and recovery expectations. External actions are exceptional risk boundaries, not the default execution path.
Learning and evolution	Evaluates LearningCandidate objects and promotes only authorized module improvements; known-material storage is separate.
Observability and recovery	Gives developers and authorized users a read-first view of ReasoningTask , RuntimeEvent , PolicyDecision , failures, and rollback options through RuntimeJournal .

6.2.2 Responsibility Layer Diagram

Figure 6.1 presents the primary responsibility structure of **MADRE**. The diagram is not a sequential request pipeline. It describes the ownership boundaries that constrain all runtime behavior.

The **MADRE** runtime kernel is the deterministic system coordinator. It owns access-surface entry, module registration, Core Module assignment, module routing, scheduling, resource assignment, global policy boundaries, **RuntimeJournal** coordination, durable runtime state, and lifecycle boundaries. The kernel does not become the reasoning actor for ordinary tasks. It selects, activates, limits, and observes reasoning modules.

Reasoning modules are the runtime units that perform domain-bounded reasoning. The module assigned the Core Module role is the first receiver for ordinary user interaction because its registered capability covers interaction continuity, clarification, general planning, and delegation. Other modules may specialize in research, knowledge handling, code, media, resource optimization, or domain-specific work. A module may answer quickly, create delayed work, invoke internal actions, or evaluate outcomes when its local policy and assigned resources allow it.

Executable behavior is not exposed as arbitrary external tools. The normal execution path uses an Agent Action boundary composed of typed runtime-defined functions with declared inputs, outputs, side-effect class, policy requirements, **RuntimeEvent** trace obligations, and recovery expectations. The first reference implementation may implement these actions on a JVM runtime, but JVM binding is not an architecture-level authority. External services, host commands, provider-backed tools, remote model services, or arbitrary plugin surfaces are not part of the default path and require stronger authority boundaries.

The Local Model Runtime Boundary is useful product-level phrasing for the point below the module and action layer where allowed inference executes. It is realized through **ModelBinding**, model-use profile, model backend, **RuntimeEvent** evidence, and generated material; it is not a chatbot server or an authority. Generated material returns to the module as generated material and is not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself.

6.2.3 Concurrent Agent Lanes

MADRE separates immediate interaction from deeper reasoning, but this separation is not modeled as two global subsystems. It is a possible execution pattern for a module-owned **MADREAgent**.

A **MADREAgent** with a **ComplexAgent** profile may maintain three concurrent lanes. The foreground lane preserves conversational continuity, answers only when safe known context is sufficient, asks for clarification when intent or risk is unclear, and creates deferred work when more evidence is required. The reasoning lane performs planning, decomposition, evidence gathering, action invocation, verification, and result construction under module authority. The reflection lane reviews recent work, observes delayed outcomes, detects failure patterns, and proposes evaluated **LearningCandidate** objects.

This pattern may exist in any reasoning module. The Core Module may use it for general interaction and delegation. A research module may use it for evidence comparison and verification. A knowledge module may use it for extraction, provenance checks, correction, and deletion semantics. The kernel schedules and constrains these lanes through module routing, resource limits, durable state, policy, and **RuntimeJournal**, but the reasoning behavior itself belongs to module-local agents.

6.3 Runtime Data Flow

Runtime data flow in **MADRE** is concurrent, governed, and module-centered. A local request enters through an access surface and is submitted to the kernel. The kernel records the request, applies the

relevant entry constraints, resolves an explicit valid module target when one exists, and otherwise falls back to the Core Module role. Semantic delegation to a more specialized module belongs to module-level reasoning, not to model preference or prompt routing.

Within the selected module, its Main Agent role may produce an immediate foreground response, create or continue delayed reasoning work, invoke internal actions, request inference through an allowed **ModelAction**, and reflect on recent outcomes. These activities are coordinated inside a module-owned agent and constrained by module policy, kernel policy, **ModelBinding**, resource assignment, durable state, **RuntimeJournal**, and recovery rules.

The following subsections describe the canonical request narrative, the governed model/action sequence, and the data-authority rules that prevent user input, source material, generated material, action results, or learning candidates from promoting themselves into authority-bearing state.

6.3.1 Canonical Request Narrative

The canonical user request begins when an access surface submits a local request to the kernel. The kernel does not interpret the request as a **KnowledgeRecord**, factual authority, or policy. It records a **RuntimeEvent**, applies entry constraints, and routes the work to the Core Module unless another module is already selected by session state, policy, or module registry evidence.

The receiving module evaluates the request through its module-local Main Agent role. If a configured foreground lane has enough safe local context, it may produce an immediate answer. If intent, scope, or risk is unclear, it asks for clarification. If the request requires evidence, stronger reasoning, internal actions, model inference, or broader context, the reasoning lane creates or continues a **ReasoningTask**. Reflection may run after the response or after delayed outcomes to propose a **LearningCandidate**. Knowledge-boundary change is separately represented as a **KnowledgeCandidate**.

Every significant step emits a **RuntimeEvent** through **RuntimeJournal**. Context bundles, actions, **RuntimeEvent** evidence, materialized generated artifacts, **PolicyDecision** outcomes, failures, correction, rollback, invalidation, supersession, and deletion/detachment remain inspectable through read-first surfaces.

6.3.2 Governed Model And Action Sequence

When module work requires executable behavior, the module invokes an internal typed action. The default action mechanism is local, typed, runtime-defined, policy-gated, and traceable. It is not an open plugin socket and not a network-exposed external action protocol.

A **ModelAction** is an **AgentAction** that uses inference. Before it runs, the module supplies a governed **ContextBundle**, an allowed **ModelBinding** and model-use profile, and the boundary authorization needed for that execution. A **PolicyDecision** records the outcome of that boundary check; it is not the executing authority. A model backend executes inference without allowing a provider, template, or runtime format to define **MADRE**'s authority model. One execution is captured as **RuntimeEvent** evidence; its generated material is produced material.

Action results may be observations. Generated material is produced material and is not a candidate by itself. It may inform a next reasoning step or undergo explicit handling into another record, but it does not authorize actions, create policy, become memory or knowledge, change routing or behavior, or promote learning. Each such transition requires its relevant authority boundary, verification path, **RuntimeEvent** record, and recovery expectation.

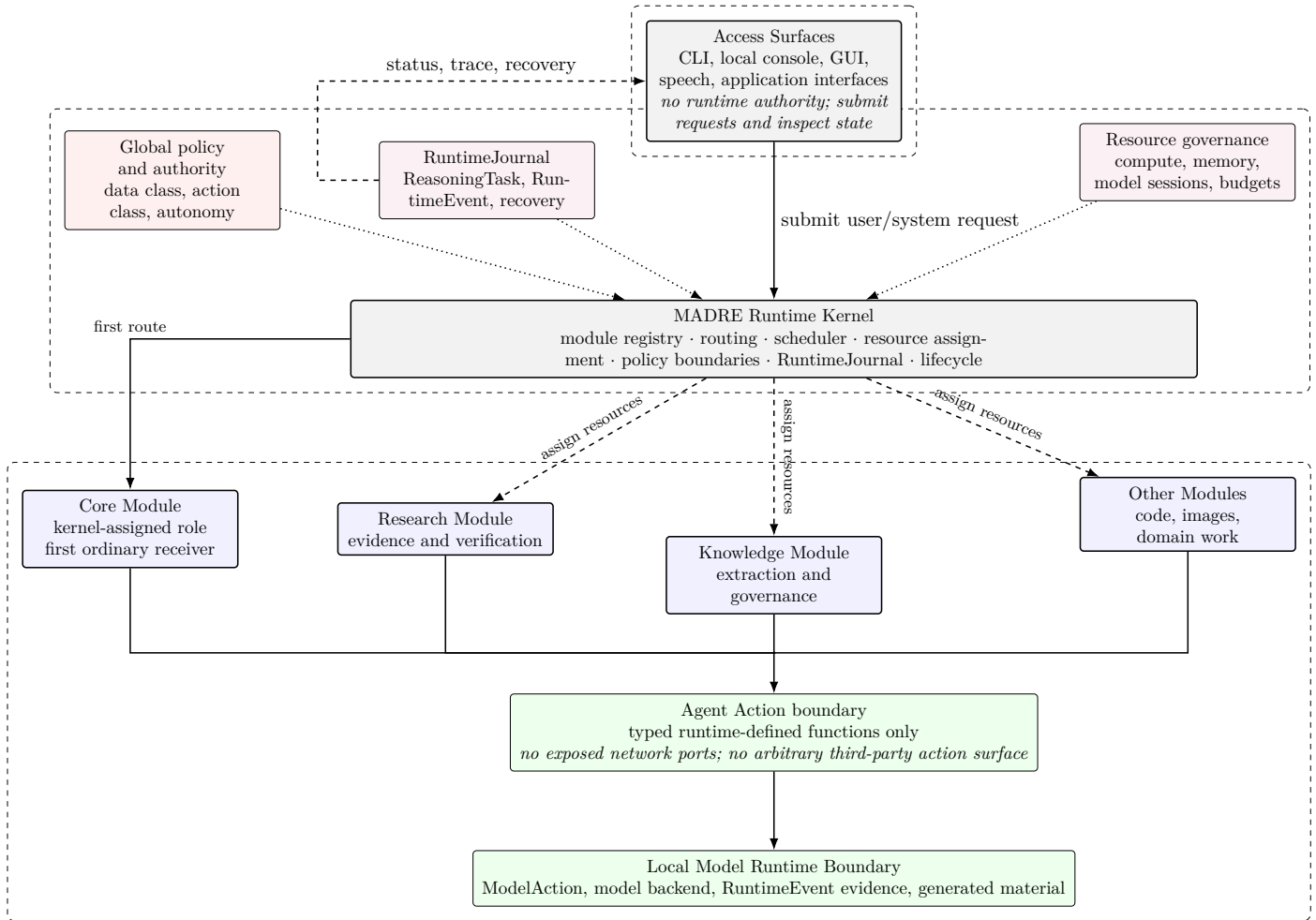


Figure 6.1: MADRE responsibility layers: kernel coordination, Core Module assignment, runtime journal, actions, and inference.

Figure 6.1 defines the top-level responsibility model. Access surfaces are intentionally outside runtime authority: they submit requests and inspect state, but they do not own policy, knowledge, routing, scheduling, or action-authority decisions. The kernel is the only system-level coordinator. It registers modules, assigns the Core Module, resolves valid targets, assigns resources, coordinates global boundary checks, records through `RuntimeJournal`, and maintains durable lifecycle state.

The module layer is the reasoning layer. The Core Module is shown first because ordinary interaction is routed there by registration, not because it is outside the module system. Research, knowledge, code, media, or domain modules are peers selected when the registry, policies, and current reasoning plan indicate a better fit. Internal actions and inference appear below the module layer because they are invoked by authorized module behavior, not by access surfaces or generated material directly.

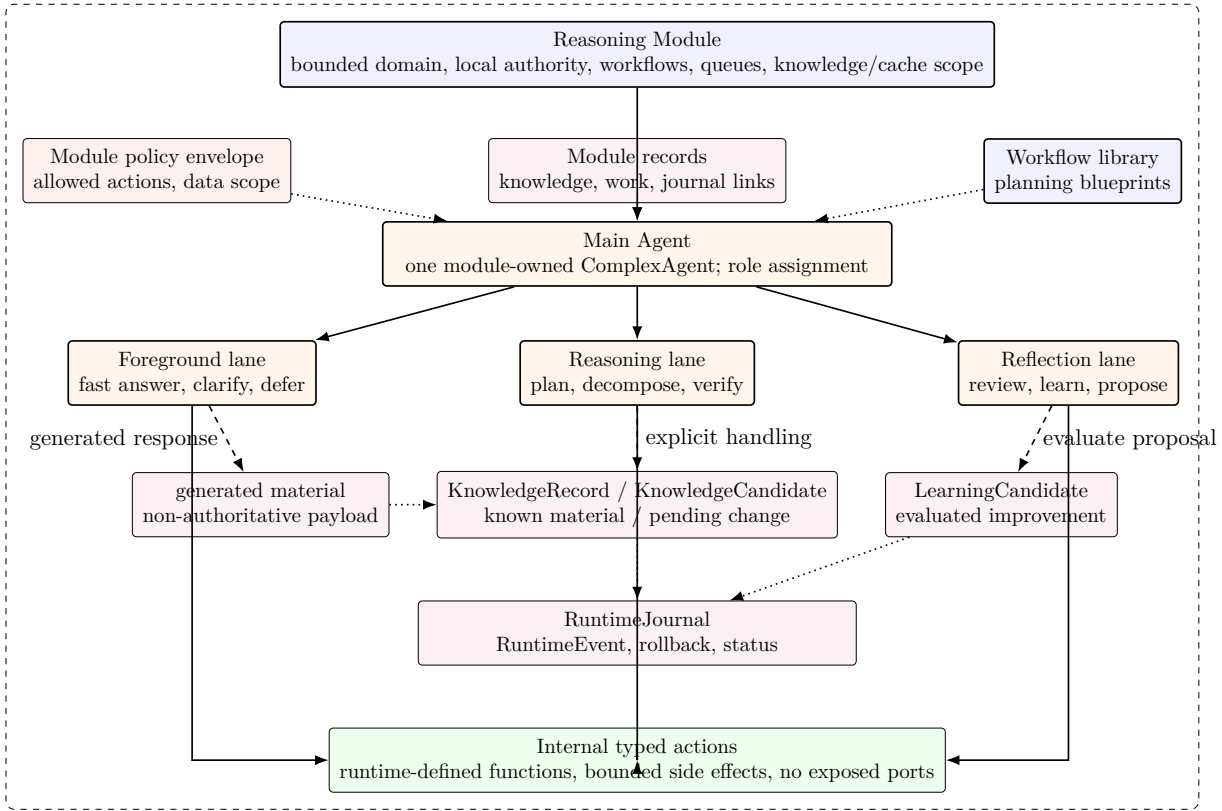


Figure 6.2: Reasoning module internal structure with its Main Agent using a ComplexAgent lane profile.

Figure 6.2 expands the internal structure of a reasoning module. A module owns a bounded domain, local policy envelope, records, workflow library, knowledge boundary, and concrete **MADREAgent** objects. One module-owned **SystemAgent** holds the Main Agent role; additional **ComplexAgent** objects may serve other module work without that role.

The foreground lane, reasoning lane, and reflection lane may operate concurrently. The foreground lane protects user continuity. The reasoning lane protects evidence, decomposition, verification, and stronger work. The reflection lane protects adaptation by evaluating a proposed **LearningCandidate** rather than silently rewriting runtime behavior. These lanes do not increase agent authority. They remain constrained by module policy, kernel policy, assigned resources, trace requirements, and promotion boundaries.

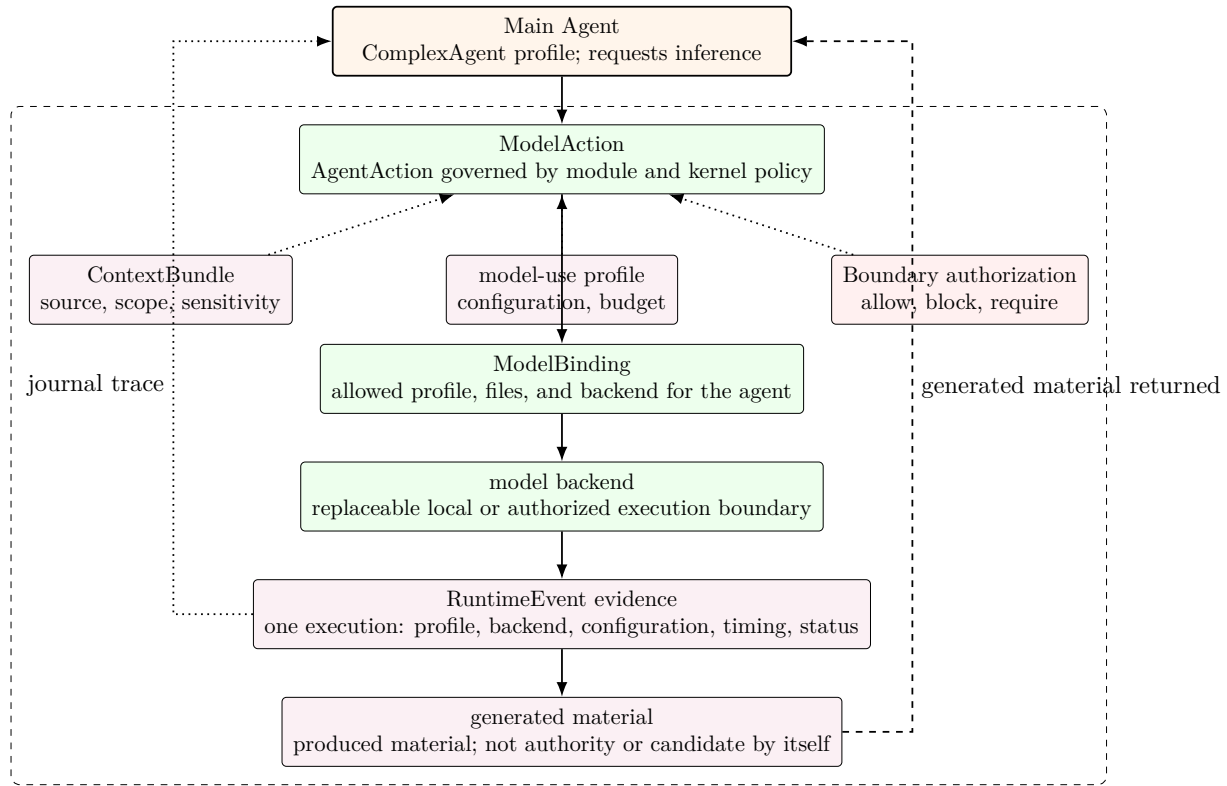


Figure 6.3: ModelAction inference path from governed execution through RuntimeEvent evidence to non-authoritative generated material.

Figure 6.3 describes inference as the governed **ModelAction** path rather than as an external server call or chat-completion product boundary. A module-owned **MADREAgent**, including one serving as Main Agent role, can request generation, critique, classification, or verification only under its allowed **ModelBinding** and a valid boundary authorization; the resulting **PolicyDecision** is the recorded outcome.

The model-use profile captures execution configuration, and the replaceable model backend executes the permitted inference. Every execution creates **RuntimeEvent** evidence; the generated material attached to that execution is produced material. The Local Model Runtime Boundary remains useful as product wording for this controlled local or authorized execution boundary, not as a competing runtime class. Generated material is not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself. Explicit module handling can retain it as retained interaction material, save appropriately classified material as **KnowledgeRecord**, evaluate it toward a **LearningCandidate**, or discard it, with the transition recorded in **RuntimeJournal**.

6.3.3 Data And Authority Rules

- User input enters as retained interaction material when retained, not as policy, knowledge, truth, action authority, or model authority.
- Retrieved, extracted, or saved material becomes **KnowledgeRecord** only with provenance, scope, sensitivity, intended use, **truthLevel**, **truthAuthority**, correction, and lifecycle metadata; it is not factual truth by default.
- Context bundles are constructed, minimized, classified, scoped, and revocable before they reach an

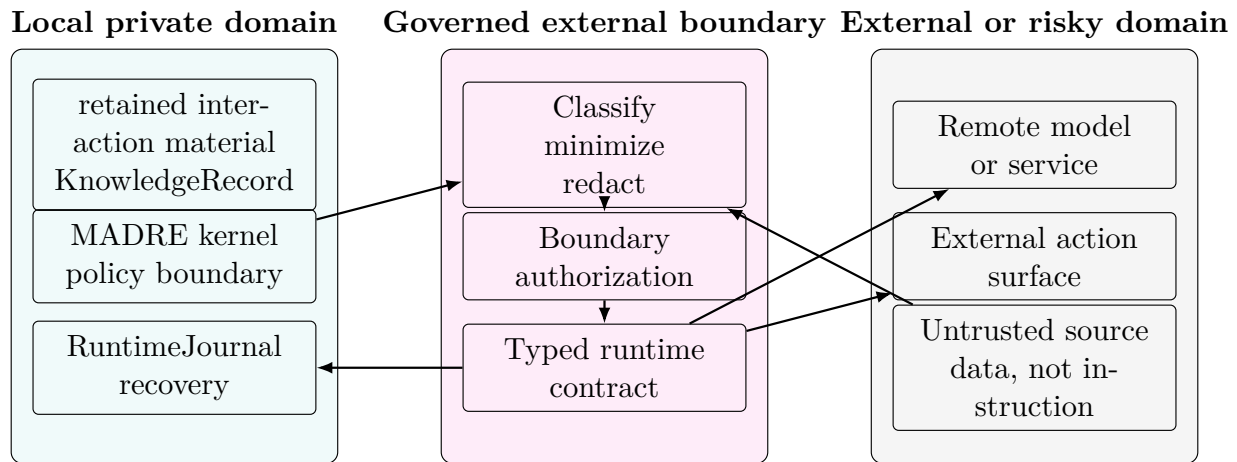
agent lane, internal action, or `ModelAction`.

- Generated material is produced material from `RuntimeEvent` evidence; it does not become an answer, policy decision, memory, knowledge, behavior, routing, authority, or candidate by itself.
- Internal action results enter as bounded observations. They become relied-on evidence only according to the action contract, verification path, and `RuntimeEvent` trace.
- External action surfaces, host commands, remote services, arbitrary filesystem mutation, credentials, and irreversible effects require stronger authority boundaries than ordinary internal actions.
- A `KnowledgeCandidate` represents only pending knowledge-boundary change; a `LearningCandidate` represents evaluated proposed module improvement and cannot directly rewrite active runtime behavior.

6.3.4 Trust Boundary Diagram

The trust boundary diagram distinguishes the ordinary local governed domain from optional external or risky domains. The default execution path remains inside the local governed domain: access surfaces, kernel coordination, reasoning modules, internal typed actions, local model runtime, `RuntimeJournal`, and recovery.

The external boundary exists for exceptional external actions: remote services, provider-backed models, host-command execution, arbitrary filesystem mutation, network access, untrusted source acquisition, or third-party integrations. These are not ordinary internal actions. They require classification, minimization, explicit policy authorization, recorded trace, and a recovery or blocking expectation appropriate to their risk class.



6.3.5 Access Surfaces

MADRE exposes access surfaces without allowing those surfaces to define the kernel. The `madre` CLI submits local requests and exposes status or result evidence. A local read-first inspection console may expose sessions, queued work, context bundles, policy outcomes, model evidence, internal action evidence, `KnowledgeCandidate`, `LearningCandidate`, `RuntimeJournal` traces, recovery, and runtime state. These surfaces operate under the same policy and authority model as every other runtime interaction.

The inspection console is not a privileged mutation path. Its transport is intentionally not fixed at architecture level. If a later implementation uses HTTP, sockets, embedded UI, or another mechanism,

that choice must be justified by deployment and security decisions and must preserve local-first, read-first, policy-governed behavior.

Graphical, speech, product-specific, or integration surfaces remain access surfaces over the runtime rather than privileged paths around it.

Safety And Evolution Policy

7.1 Runtime Safety

MADRE is intended to run useful delayed work while the user is not constantly present. That requirement makes safety an architectural property, not a documentation afterthought. The system must be able to explain what it did, why it did it, what data it used, what boundary allowed it, and how the result can be corrected or rolled back when the action is reversible.

7.1.1 Policy Before Action

The kernel must not expose executable behavior merely because a model can request it. Internal actions, external calls, knowledge-boundary changes, workflow application or mutation, and learning promotion need a valid boundary outcome recorded as **PolicyDecision** before protected state changes. The safe path should be the normal path, and safe autonomous improvement should not require constant user interruption. Unsafe paths should be technically isolated, traceable, recoverable where possible, and visible to the responsible developer or operator through **RuntimeJournal**.

Typed runtime-defined internal actions represent the canonical bounded action form. They may be allowed only through a policy-gated action boundary. Remote action, host commands, arbitrary filesystem access, mutation actions, provider-backed integrations, active learning promotion, and workflow mutation require stronger authority boundaries than bounded internal action execution.

7.1.2 Autonomy Levels

Level	Meaning
Observation	Read local runtime state, existing knowledge, and safe metadata without mutation.
Draft creation	Produce summaries, candidate records, or proposed changes without promoting them into active state.
Bounded mutation	Modify authorized local state only when the action is logged, scoped, reversible where possible, and policy-permitted.
External action	Contact remote systems, use external services, or disclose data only after classification, minimization, authorization, provenance, and recorded trace.
Critical action	Credentials, identity, legal, financial, destructive, irreversible, or executable-code actions require a stronger human authorization path.

7.2 Evolution Control

7.2.1 Evolution Rules

- Runtime learning is represented by evaluated **LearningCandidate** proposals, not only by manual feedback collection.
- Learning strategies must declare what they are allowed to observe, propose, mutate, verify, and promote.
- A **LearningCandidate** cannot silently modify protected behavior.
- Learning proposals begin with source, evaluation evidence, scope, and rollback information before they are promoted.
- A **KnowledgeCandidate** is used only for pending knowledge-boundary change and requires provenance, truth metadata, correction paths, and recovery options.
- Saving a **KnowledgeRecord** is not learning by itself.
- Workflow and routing improvements may become default behavior through developer-defined checks, evaluation gates, and rollback policies, without requiring user authorization for every routine improvement.
- Model adaptation, fine-tuning, or weight updates are advanced model-evolution actions governed by stronger model-adaptation authority.
- Critical learning failures must be recoverable through snapshots, versioned state, VCS-backed records, or an equivalent rollback mechanism.

This policy keeps **MADRE** capable of evolving through conscious, checked learning without reducing improvement to constant user authorization. The final user should be interrupted as little as possible. Routine learning should be governed by developer-defined strategies, evidence, tests, promotion gates, **RuntimeJournal** traces, and rollback mechanisms; user authorization is reserved for boundaries where risk, privacy, identity, external action, or policy explicitly requires it.

D-011 fixes the controlled-promotion rule: **KnowledgeCandidate** denotes pending knowledge-boundary change and **LearningCandidate** denotes evaluated proposed module improvement. Neither candidate creation nor **KnowledgeRecord** storage silently rewrites protected behavior.

System Engineering Baseline

8.1 Engineering Problem

MADRE addresses the engineering problem of coordinating model inference without allowing the model to become the system. The runtime must support foreground continuity, delayed reasoning, governed context, bounded internal actions, known material, learning, runtime traceability, and recovery while preserving local ownership of data and operational control.

The core difficulty is not only model accuracy. A useful local assistant must decide when an answer is safe in the foreground lane, when work must be delayed, which context may be exposed, which internal action may be invoked, which result requires verification, and how a mistaken or unsafe outcome can be inspected, corrected, or rolled back.

8.2 Stakeholders And Concerns

MADRE serves users who need private assistance, developers who need an extensible runtime, organizations that need governed local knowledge, and maintainers who must keep the system auditable and evolvable. Each stakeholder depends on the same architectural guarantees: model independence, local-first operation, explicit authority boundaries, understandable execution, and recoverable state.

The principal concerns are privacy, cost, autonomy control, model substitution, context minimization, data provenance, action safety, knowledge correction, learning control, operational continuity, and evidence for system behavior. A runtime that cannot expose those concerns as inspectable state cannot satisfy **MADRE**'s product objective, even if its conversational output appears fluent.

8.3 Requirement Domains

MADRE's functional baseline is organized around runtime domains. The foreground lane domain preserves conversational flow, asks for clarification, and creates deeper work when the answer requires evidence or internal actions. The delayed reasoning domain records work durably, schedules it, verifies results, and returns outcomes without erasing the foreground interaction.

The context domain selects, minimizes, classifies, scopes, and revokes information before it reaches models or internal actions. The inference domain uses **ModelAction**, permitted **ModelBinding**, model-use profile, and replaceable model backend contracts. The internal action domain exposes declared side effects under **PolicyDecision** and **RuntimeEvent** recording. The knowledge domain separates retained interaction material, **KnowledgeRecord**, pending **KnowledgeCandidate**, untrusted source material, and evaluated **LearningCandidate**.

The learning domain keeps improvement proposals quarantined until evidence, evaluation, promotion, and rollback rules allow activation. The observability domain exposes sessions, **ReasoningTask** lifecycle, context bundles, **RuntimeEvent** evidence, actions, **PolicyDecision**, failures, metrics, and recovery paths. The deployment domain keeps ordinary local operation viable before optional remote integrations are considered.

8.4 Traceability Surface

Each significant product claim in **MADRE** must be visible as system behavior. A claim such as local-first operation corresponds to local execution paths, storage boundaries, remote-transfer controls, and inspection evidence. A claim such as model agnosticism corresponds to replaceable model-use profile and model backend contracts and tests that prevent provider-specific assumptions from entering the kernel.

The same trace applies to safety claims. Policy before action must appear as an enforceable boundary around internal actions, external calls, knowledge-boundary change, workflow application or mutation, and learning promotion. Governed context must appear as provenance, classification, minimization, scope, revocation, and journaled trace. Delayed reasoning must appear as **ReasoningTask**, **RuntimeEvent** transitions, verification, result delivery, and recovery.

8.5 Quality Attributes

MADRE prioritizes data sovereignty, privacy, resource discipline, responsiveness, reliability, recoverability, maintainability, portability, auditability, and resistance to hallucination-driven action. These attributes are not independent decorations; they define whether the runtime fulfills its purpose.

Responsiveness is measured by preserving useful foreground conversation while heavier work is delayed. Reliability is measured by durable records, repeatable status, bounded retries, and visible failures. Maintainability is measured by the ability to replace models, internal actions, workflows, and interfaces without rewriting the kernel. Auditability is measured by the ability to explain what context, **RuntimeEvent** evidence, action, **PolicyDecision**, and verification path produced an outcome through **RuntimeJournal**.

8.6 Risk And Policy Boundaries

MADRE treats prompt injection, data leakage, excessive agency, unsafe action execution, poisoned knowledge, provider dependency, remote retention, unbounded consumption, credential exposure, and rollback gaps as architectural risks. The runtime must assume that generated material can be persuasive, wrong, incomplete, unsafe, or maliciously influenced by context.

Risk control is expressed through policy boundaries. Data classification governs what may be retrieved, stored, disclosed, or sent to a model. Autonomy level governs whether a result may remain a draft, update local state, invoke an internal action, contact an external service, or require stronger authorization. Recovery expectation governs whether the action must be reversible, snapshot-backed, compensatable, or blocked.

Requirements Baseline

9.1 Requirement Taxonomy

Prefix	Requirement family
FR	Functional behavior of foreground-lane conversation, delayed reasoning, context construction, inference, internal actions, knowledge, learning, runtime trace, and recovery.
NFR	Quality attributes such as responsiveness, reliability, recoverability, maintainability, portability, resource discipline, and auditability.
SEC	Security and trust controls, including prompt injection resistance, credential protection, action authority, and remote-transfer boundaries.
DATA	Data, knowledge, provenance, retention, deletion, correction, revocation, and quarantine requirements.
AI	ModelAction, model substitution, generated material handling, and verification requirements.
UX	User-visible states, clarification, authorization, progress, failure, correction, and result-delivery requirements.
OPS	Local deployment, storage, backup, diagnostics, migration, and operational recovery requirements.

9.2 Functional Requirements

ID	Requirement	Acceptance criteria	Verification
FR-001	The runtime shall preserve foreground-lane conversation while delaying evidence-heavy, risky, action-dependent, or broad-context work.	Immediate answers stay within safe known context; stronger work is recorded as delayed work.	Triage scenarios; D-001, D-004.
FR-002	The runtime shall represent delayed work as ReasoningTask objects with inspectable status and recovery path.	Work exposes queued, running, blocked, failed, completed, and recovered states with RuntimeEvent history in RuntimeJournal .	Work-lifecycle and recovery scenarios; D-001, D-003, D-008.
FR-003	The runtime shall build context bundles from sourced, classified, minimized, scoped, provenanced, and revocable material.	Context is validated or rejected before ModelAction or internal action use.	Context-governance scenarios; D-001, D-005, D-007.
FR-004	The runtime shall execute model-backed work through ModelAction constrained by ModelBinding and replaceable model-use profile/model backend contracts.	Every execution produces an inspectable RuntimeEvent evidence and keeps its generated material non-authoritative.	Inference substitution and output-boundary scenarios; D-001, D-006.
FR-005	The runtime shall expose executable behavior only as bounded internal actions or explicitly authorized external actions.	Internal actions remain policy-gated, while disallowed action requests are blocked or authorization-gated.	Action policy scenarios; D-007, D-009.
FR-006	The runtime shall keep KnowledgeCandidate and LearningCandidate semantics separate and inactive unless their respective promotion rules activate them.	Saving a KnowledgeRecord does not activate learning; evaluated behavior, routing, workflow, or inference-selection improvement remains a LearningCandidate until promoted.	Knowledge-boundary and learning-promotion scenarios; D-011.

ID	Requirement	Acceptance criteria	Verification
FR-007	The runtime shall expose local request entry and read-first inspection.	Local CLI request/status interaction and local console inspection expose system state without bypassing runtime authority.	Local access and inspection scenarios; D-010.
FR-008	The runtime shall represent recovery outcomes explicitly.	Correction, rollback, deletion/detachment, invalidation, and supersession are distinguishable and auditable.	Recovery readout scenarios; D-003, D-008.

9.3 Cross-Cutting Requirements

ID	Requirement	Rationale	Verification
NFR-001	Local useful operation shall remain possible without mandatory remote service dependency.	Data sovereignty, cost control, and resilience.	Local execution evidence; D-001, D-002, D-006.
NFR-002	Runtime state changes shall be inspectable through local read models.	Auditability and recoverability.	Inspection review; D-003, D-010.
NFR-003	Delayed work shall be bounded by status, resource, and failure discipline.	Prevents unbounded cost, compute, storage, and queue growth.	Resource/status scenarios; D-003, D-007, D-010.
SEC-001	Prompt or source material shall not become instruction, policy, or authority without explicit classification and boundary checks.	Source-as-instruction injection risk.	Adversarial context scenario; D-004, D-005, D-007.
SEC-002	A <code>PolicyDecision</code> shall explicitly represent allow, block, or authorization-required before action execution or promotion and be linked to <code>RuntimeEvent</code> .	Prevents hidden authority decisions and excessive agency.	Policy matrix review; D-007, D-009, D-011.

ID	Requirement	Rationale	Verification
SEC-003	External, destructive, credential, legal, financial, executable, or irreversible effects shall be blocked or separately authorized.	Preserves recoverability and explicit authority over critical action.	Critical-action negative paths; D-007, D-008, D-009.
DATA-001	A KnowledgeRecord shall carry provenance, scope, sensitivity, intended use, truthLevel , truthAuthority , correction, and lifecycle metadata; it is not factual truth by default.	Knowledge governance and user control.	Non-authoritative known-material, correction, and revocation scenarios; D-005, D-008, D-011.
DATA-002	Active state and accountable RuntimeEvent history shall remain distinguishable.	Allows deletion/detachment without losing required trace history.	Recovery inspection scenarios; D-003, D-008.
AI-001	Generated material shall remain produced material recorded by RuntimeEvent evidence, not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself.	Hallucination and false-authority risk.	Output non-authority and non-candidate scenarios; D-001, D-006.
AI-002	Models and generated material shall not create or widen PolicyDecision , authorize actions, establish knowledge truth, alter behavior or routing, or activate learning.	Keeps runtime authority outside generated material.	Model-overreach and promotion-control scenarios; D-006, D-007, D-011.
UX-001	Users shall be able to distinguish immediate answers, queued work, running work, blocked work, failed work, completed work, and authorization requests.	Continuity and user control.	Interaction-state review; D-004, D-010, D-011.

ID	Requirement	Rationale	Verification
OPS-001	Local access surfaces shall submit requests and inspect <code>RuntimeJournal</code> status and trace without becoming runtime authority.	Supports operability while preserving governance.	Local access and inspection scenario; D-010.

Traceability Model

10.1 Traceability Method

Traceability in **MADRE** links the product claim, stakeholder concern, requirement, architecture obligation, validation method, and evidence status. The purpose is to keep evaluation anchored to the system thesis: local-first operation, model agnosticism, delayed reasoning, governed context, bounded internal actions, evaluated learning promotion, runtime traceability, and recovery.

10.2 Architecture Traceability Matrix

Claim	Concern	Requirement	Decision trace	Evidence family
Local-first operation	Data sovereignty, cost, resilience	NFR-001	D-001, D-002, D-010	Local request, local status, local journal trace, no mandatory remote service.
Model agnosticism	Provider independence	FR-004, AI-001	D-006	Replaceable model backend/model-use profile substitution plus generated material non-authority scenario.
Delayed reasoning	Responsiveness and quality	FR-001, FR-002	D-001, D-003, D-004	Foreground triage plus durable delayed-work lifecycle.
Governed context and known material	Privacy and correctness	FR-003, DATA-001	D-005, D-007	Context acceptance, source-as-data, non-authoritative KnowledgeRecord use, truth-metadata correction, and revocation scenario.

Claim	Concern	Requirement	Decision trace	Evidence family
Bounded internal actions	Safety and recoverability	FR-005, SEC-001	D-007, D-009	Policy-gated internal action and blocked unsafe external action attempts.
Learning promotion control	Controlled evolution	FR-006	D-011	LearningCandidate evaluation and inspection without active promotion; KnowledgeRecord save is not treated as learning.
Runtime trace and recovery	Accountability and correction	FR-008, NFR-002, DATA-002	D-003, D-008, D-010	RuntimeJournal/Runtime status, correction, invalidation, supersession, roll-back, and deletion/detachment trace.

10.3 Integrated Architecture Trace

Step	Required behavior	Decision trace	Negative evidence
Local entry	Request enters through local access surface.	D-001, D-010	Internal-only execution is not enough to prove inspection.
Foreground triage	Foreground answers only safe known context, clarifies, blocks, or creates delayed work.	D-004, D-007	Evidence-heavy request is not answered as if already known.
Durable work	Delayed work is represented by ReasoningTask with RuntimeEvent status history.	D-003	Work is not lost or silently overwritten.

Step	Required behavior	Decision trace	Negative evidence
Governed context	Context is sourced, classified, minimized, scoped, provenanced, revocable, and linked to journal trace.	D-005, D-007	Sensitive, unrelated, revoked, or incorrectly truth-classified material is excluded or blocked.
Inference boundary	<code>ModelAction</code> uses model backend, creates <code>RuntimeEvent</code> evidence, and returns non-authoritative generated material.	D-006	Generated material does not become authority or a candidate by itself.
Bounded internal action	Typed runtime-defined internal actions are policy-gated and recorded.	D-009	Remote, host-command, filesystem, or mutation actions require stronger authority boundaries.
Knowledge and learning handling	<code>KnowledgeCandidate</code> covers pending knowledge-boundary change; <code>LearningCandidate</code> covers evaluated improvement.	D-011	<code>KnowledgeRecord</code> storage does not activate learning and candidates cannot promote themselves.
Inspection and recovery	Status, trace, failure, correction, rollback, invalidation, supersession, and deletion/detachment are visible through <code>RuntimeJournal</code> .	D-008, D-010	Active state changes do not erase accountable history.

Design Decision Traceability

Design decisions D-001 through D-011 provide traceability from product requirements to architecture obligations and validation evidence. They identify accepted product architecture decisions for governed delayed reasoning, durable work, context, inference, policy, recovery, bounded internal actions, local access, knowledge-boundary control, and evaluated learning promotion.

Decision	Product-level effect	Architecture boundary
D-001	Defines source-backed delayed-answer behavior as a core expression of governed runtime value.	Conversation remains connected to durable work and evidence.
D-002	Establishes behavior-first architectural validation as part of the product's engineering discipline.	Observable behavior anchors architecture claims.
D-003	Requires ReasoningTask lifecycle and projected inspection state.	RuntimeJournal and append-only RuntimeEvent preserve local traceability.
D-004	Requires conservative foreground triage.	Unsupported answers become clarification, policy block, or delayed work.
D-005	Requires governed context acceptance and rejection.	Context is sourced, classified, minimized, scoped, provenanced, and revocable.
D-006	Requires ModelAction , model backend, RuntimeEvent evidence, and non-authoritative generated material.	Models remain runtime functions, not authorities or candidate creators by generation alone.
D-007	Requires explicit PolicyDecision before action execution.	Action execution follows policy rather than model instruction.
D-008	Requires recovery as a recorded semantic transition.	Correction, rollback, invalidation, supersession, deletion, and detachment have distinguishable RuntimeEvent meaning.

Decision	Product-level effect	Architecture boundary
D-009	Defines bounded internal actions as the low-risk executable reference.	Action execution is policy-gated, traceable, and recoverability-aware.
D-010	Defines CLI request/status interaction and local read-first console inspection as canonical local access surfaces.	Access surfaces expose the kernel without becoming runtime authority.
D-011	Distinguishes KnowledgeCandidate from LearningCandidate improvements.	pending boundary changes from evaluated module Known-material storage is not learning; proposed evolution remains inspectable until promoted by authority.

Architecture Viewpoints

12.1 Viewpoint Catalogue

Viewpoint	Primary concern	MADRE content
System context	Runtime boundary and environment	User, local device, filesystem, operating system, local models, optional external integrations, internal actions, and untrusted sources.
Logical	Responsibility separation	Kernel, Core Module, module-owned agents, scheduler, context/knowledge, Local Model Runtime Boundary, Agent Action boundary, PolicyDecision, RuntimeJournal, learning, and recovery.
Runtime/process	Work lifecycle	User turn, triage, ReasoningTask, context bundle, scheduling, RuntimeEvent evidence, result delivery, RuntimeEvent, and recovery.
Data/knowledge	Data ownership and provenance	retained interaction material, KnowledgeRecord, KnowledgeCandidate, LearningCandidate, truth metadata, retention, correction, and deletion.
Security/trust	Boundary enforcement	Local domain, governed external boundary, external/risky domain, prompt injection, remote transfer, credential exposure, and action side effects.

Viewpoint	Primary concern	MADRE content
Deployment	Local operation	Ordinary device profile, local storage, model availability, backup, migration, diagnostics, and optional remote configuration.
Extension	Developer evolution	<code>ModelBinding</code> , model-use profile, model backend, action contracts, workflows, projections, context policies, and interface surfaces.
UX	User-visible control	Conversation, clarification, queue state, progress, authorization, failure, completion, correction, and rollback.

12.2 Viewpoint Design Focus

Viewpoint	Architecture focus	Authority boundary
System context	Local user, local device, CLI, local read-first console, local sources, and optional remote integrations.	Local authority remains explicit across every access surface.
Logical	Kernel coordination over routing, delayed work, context, bounded action use, <code>PolicyDecision</code> , <code>RuntimeJournal</code> , and recovery.	Runtime authority is not delegated to interfaces, models, actions, or generated material.
Runtime/process	Governed lifecycle from local request to inspection and recovery.	<code>ReasoningTask</code> transitions remain recorded, inspectable, and recoverable.
Data/knowledge	Governed context and explicit knowledge/learning types.	Source material, retained interaction material, <code>KnowledgeRecord</code> , <code>KnowledgeCandidate</code> , and <code>LearningCandidate</code> remain distinguishable.
Security/trust	Source-as-data, model-as-function, policy before action, and bounded internal action authority.	Action execution follows declared policy outcomes.

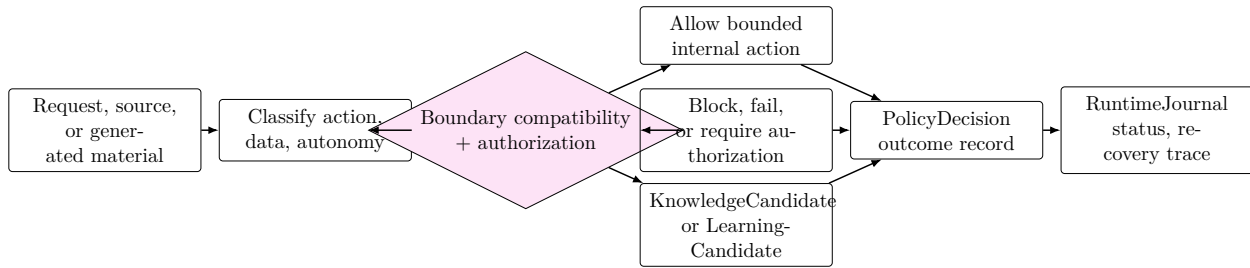
Viewpoint	Architecture focus	Authority boundary
Observability/recovery	Local status, <code>RuntimeJournal</code> , verification, policy, knowledge/learning transition, and recovery evidence.	Read-first inspection must not become a privileged mutation path.
Extension	Inference backends, actions, workflows, and learning systems inherit the defined boundaries.	Extension preserves kernel authority and product traceability.

Runtime Governance

13.1 Runtime Authority Rule

Runtime authority belongs to the constructed **MADRE** boundary path: kernel entry, module ownership, context construction, action contract, model binding, resource limit, and authorization rule. User turns, source text, generated material, action results, local inspection, **KnowledgeCandidate**, and **LearningCandidate** can inform decisions, but they cannot authorize their own authority, promote themselves, mutate protected state, or widen permissions. Bounded internal action use requires a valid boundary outcome, recorded as **PolicyDecision**; local access is read-first where it inspects state, and transitions are recorded through **RuntimeJournal**.

13.2 Policy And Action Gate



13.3 Policy Matrix

Action class			Data class		Autonomy level		Authorization	Trace/Recovery
Read	local	meta-data	Public	or low-sensitivity metadata	low-local	Observation	None by default	Log access when used for reasoning.
Draft text			Any	allowed text bundle	con-	Draft creation	None unless sensitive scope is crossed	Retain as conversation or generated material, not authority or candidate by itself.

Action class	Data class	Autonomy level	Authorization	Trace/Recovery
Update known material	Scoped module material	Bounded mutation	Required provenance, metadata, scope, or sensitivity changes when truth changes	Versioned KnowledgeRecord with correction and rollback; KnowledgeCandidate if boundary change is pending.
Invoke internal action	Small task-scoped non-sensitive input	Bounded internal action	Policy allow required	Requested action, input class, result, policy outcome, and recovery expectation recorded.
Contact remote service	Minimized and classified transfer set	External action	Explicit policy authorization	Transfer record, provider, purpose, declared retention terms where available, and any retention uncertainty recorded; sensitive transfers block when policy does not accept uncertainty.
Modify credentials, identity, legal, financial, destructive, or executable state	Sensitive or critical data	Critical action	Strong human authorization	Block unless recovery or explicit irreversible authorization exists.

Knowledge-boundary updates, routing mutation, workflow mutation, and learning lifecycle behavior follow the same authority rule: a pending knowledge-boundary change is a **KnowledgeCandidate**; an evaluated behavior improvement is a **LearningCandidate**; neither becomes active merely by being produced.

13.4 Learning Lifecycle

Learning follows a controlled lifecycle: observation, evaluation, **LearningCandidate** creation, evidence collection, evaluation, promotion, active monitoring, and rollback. Knowledge handling remains distinct: a stored **KnowledgeRecord** is not learning, and a pending change to its owning knowledge boundary is a **KnowledgeCandidate**. Routing preferences, workflow improvements, and inference-selection improvements remain inactive unless the appropriate learning promotion rule validates them for a defined scope.

Threat Model

14.1 Threat Taxonomy

Threat	MADRE exposure	Primary control	Verification surface
Source as instruction	Untrusted text attempts to alter policy, action execution, or knowledge handling by being interpreted as an instruction.	Source classification and instruction/data separation.	Adversarial context scenario.
Sensitive disclosure	Private context leaks to a model, action, or remote service.	Data classification, minimization, and remote-transfer authorization.	Disclosure review.
Excessive agency	generated material causes over-broad action.	Autonomy levels and action-class policy.	Policy matrix review.
Unsafe action execution	Action invocation produces side effects outside task scope.	Policy-gated internal action boundary; remote, host-command, filesystem, and mutation actions require stronger authority.	Allowed-action and blocked-action scenarios.
Incorrect truth authority for known material	A <code>KnowledgeRecord</code> is used as factual authority despite unsuitable <code>truthLevel</code> or <code>truthAuthority</code> .	Required knowledge metadata, intended-use checks, correction, and reclassification path.	Non-authoritative knowledge-use and truth-correction scenario.
Inspection misuse	Local console is treated as authority or mutation path.	Read-first inspection boundary and shared policy model.	Local inspection negative scenario.

Threat	MADRE exposure	Primary control	Verification surface
Supply-chain or provider dependency	Model, dependency, or service changes behavior or availability.	Replaceable model backend and dependency policy.	Inference substitution review.
Remote transfer risk	Local data is sent outside the user's domain without classification, minimization, authorization, or recorded trace.	Local-first default and remote-transfer policy boundary.	Remote-contact blocked scenario.
Unbounded consumption	Delayed work consumes excessive compute, storage, or cost.	Resource budgets, queue limits, and status inspection.	Resource benchmark.
Trace/recovery gaps	A result cannot be explained, corrected, invalidated, superseded, rolled back, or detached.	<code>RuntimeJournal</code> , append-only <code>RuntimeEvent</code> , and recovery semantics.	Recovery trace scenario.
Inference output as false authority	Fluent generated material is treated as truth, policy, knowledge, behavior, routing, authority, or a candidate by itself.	Explicit generated-material handling and recorded boundary outcome.	Output non-authority/non-candidate scenario.

Quality Model

15.1 Quality Attribute Scenarios

Attribute	Scenario	Measure or evidence
Responsiveness	The foreground remains conversational while deeper work is queued.	Foreground latency and clear work status.
Recoverability	A failed or unsafe delayed task can be inspected and corrected.	<code>RuntimeJournal</code> trace, status transition, and roll-back/correction path.
Auditability	A result can be inspected through context, <code>RuntimeEvent</code> evidence, action, <code>PolicyDecision</code> , and verification records.	Complete journaled outcome evidence chain.
Maintainability	An model backend or action contract can be replaced without changing kernel authority.	Contract and substitution review.
Portability	A useful subset runs on ordinary local hardware.	Local profile and resource benchmark.
Security	Untrusted context cannot silently become instruction or policy.	Prompt-injection and data-boundary scenarios.
Resource discipline	Delayed work is bounded by cost, time, storage, and compute policies.	Queue and resource-use metrics.
Privacy	Sensitive or unrelated local material is excluded from context and remote transfer unless scoped and policy-allowed.	Context exclusion and remote-transfer scenarios.
Model substitutability	Boundary behavior is preserved across replaceable model-use profile and model backend choices.	Inference substitution scenario.
Context isolation	One task's context does not silently bleed into another topic, user, inference execution, action invocation, or candidate.	Context scope and revocation review.

Attribute	Scenario	Measure or evidence
Explainability and inspectability	Users and developers can see status, <code>PolicyDecision</code> , context evidence, <code>RuntimeEvent</code> evidence, action evidence, candidates, and recovery state.	Local read-first inspection scenario.
Testability	Architecture claims can be checked through behavior scenarios and negative paths.	Requirements and decision trace review.

UX, Observability, And Recovery

16.1 Interaction Contract

State	User-visible meaning
Immediate answer	The foreground has enough safe local context to answer.
Clarification needed	Intent, scope, risk, or context is insufficient.
Queued reasoning	Work has been recorded for delayed execution.
Running task	Delayed reasoning or action work is active.
Blocked by policy	A policy boundary prevents execution or promotion.
Authorization required	The action crosses a configured authority boundary.
Failed task	Work ended without acceptable result or recovery.
Completed task	Result is available with <code>RuntimeJournal</code> history.
Corrected or rolled back	A previous result or knowledge-boundary change has been revised or reverted.
Invalidated or superseded	Prior evidence, source, or result is no longer active because it was corrected, revoked, reclassified, or replaced.
Deleted or detached	Active generated material is removed from use where allowed while accountable <code>RuntimeEvent</code> history remains distinguishable.
Learning candidate	An evaluated <code>LearningCandidate</code> proposal exists but is not active.

16.2 Observability Surfaces

The local inspection surface must expose sessions, `ReasoningTask`, `RuntimeEvent`, context bundles, `RuntimeEvent` evidence and generated material, internal action evidence, `PolicyDecision`, failures, metrics, checkpoints, rollbacks, invalidations, supersessions, deleted/detached active material, `KnowledgeCandidate`, `LearningCandidate`, available recovery options, and authorized recovery workflows. The inspection surface is not a privileged bypass; it is a read-first view over the same policy-governed runtime.

Evaluation And Benchmark Specification

MADRE defines an architectural evaluation surface for testing whether the runtime preserves local authority, governed context, bounded action use, evaluated learning promotion, journaled traceability, and recoverable delayed reasoning.

17.1 Benchmark Families

Benchmark family	Purpose
Functional correctness	Confirm expected runtime behavior for foreground-lane continuity and delayed work.
Delayed reasoning quality	Evaluate evidence handling, decomposition, verification, and result delivery.
Foreground triage	Confirm unsupported evidence-heavy answers are clarified or delayed rather than fabricated.
Context isolation	Confirm unrelated or sensitive knowledge does not enter the wrong bundle.
Inference substitution	Confirm replaceable model-use profile and model backend choices preserve the same boundary behavior and expose failures.
Policy enforcement	Confirm action, data, autonomy, authorization, and recovery rules are enforced.
Bounded internal action	Confirm the bounded internal action is policy-gated, recorded, constrained to declared side effects, and recoverable or explicitly marked non-recoverable.
Blocked unsafe actions	Confirm remote, host-command, filesystem, mutation, and critical-action attempts do not execute under bounded internal-action authority.
Inference output non-authority	Confirm generated material remains generated material, neither authority nor a candidate by itself.

Benchmark family	Purpose
Known-material truth control	Confirm a <code>KnowledgeRecord</code> with non-authoritative truth metadata is not used as factual truth.
Learning promotion control	Confirm a <code>LearningCandidate</code> remains inactive until evaluated promotion and that saving <code>KnowledgeRecord</code> is not learning.
Runtime journal traceability	Confirm <code>ReasoningTask</code> , <code>RuntimeEvent</code> , <code>PolicyDecision</code> , and inference references expose lifecycle and recovery evidence locally.
Read-first observability	Confirm CLI/status and local console inspection expose lifecycle, policy, context, inference, action, knowledge/learning transition, and recovery evidence.
Recovery	Confirm failed or unsafe outputs can be inspected, corrected, deleted/detached, invalidated, superseded, or rolled back.
Resource use	Measure latency, storage, queue time, model cost, and local compute use.
Reproducibility	Preserve enough configuration, context, and event history to review outcomes.
Hallucination resistance	Confirm generated material does not become truth, knowledge, action authority, routing, behavior, or candidate status by itself.

Governance, Deployment, And Limits

18.1 Secure Engineering Principles

MADRE requires secure defaults for local credentials, dependency selection, governed external boundaries, secrets handling, reviewable release records, and vulnerability response. Security evidence must cover both conventional software risks and model-mediated risks such as prompt injection, poisoned retrieval, unsafe output handling, excessive agency, and remote disclosure.

18.2 Open-Source Governance

MADRE is released under GPL-3.0. The open-source governance model must preserve license clarity, dependency review, contribution rules, security disclosure, release integrity, and community responsibility. Openness is part of the project value only when users and developers can inspect, adapt, and verify the runtime without surrendering private data to a platform owner.

18.3 Deployment And Operational Assumptions

The baseline deployment target is an ordinary local device with local storage, local configuration, optional local model execution, and optional remote integrations governed by policy. The operational profile includes storage paths, model setup assumptions, backup and restore, migration, diagnostics, log retention, offline behavior, and upgrade or rollback expectations.

Deployment assumptions remain conceptual at this level of specification. The product architecture defines authority, locality, inspection, recovery, model-use boundaries, and action boundaries without reducing deployment to a single operational shape.

18.4 Risks, Limitations, And Excluded Functions

MADRE limits active executable behavior by authority class. Workflow authoring, self-learning, speech interaction, graphical product surfaces, distributed inference, organization-scale deployment, and model fine-tuning remain governed by the same local-first, policy-governed, model-agnostic, auditable, and recoverable runtime boundary as the rest of the product.

Runtime Domain Model

MADRE principles are rooted in the domain architecture rather than brute computation. The runtime class map below is intentionally conservative: a class appears only when it owns stable identity, lifecycle, ownership boundary, responsibility, canonical-flow participation, or boundary participation that cannot safely be collapsed into another object.

This chapter incorporates the accepted classmap baseline. It removes storage adapters, inference records, inference outputs, generic boundary-contract containers, and conversational record classes from the runtime domain map. Those concepts remain possible infrastructure, product wording, retained material, or event payloads, but they are not accepted runtime classes here.

19.1 Design Rule

A class exists in this map only when it has stable runtime identity, stable ownership boundary, stable lifecycle, stable responsibility that cannot be expressed as a value inside another class, required participation in the canonical **MADRE** flow, or boundary participation needed for valid composition. Attributes preserve identity, ownership, lifecycle, relation, traceability, or accepted invariants. Operations appear only when caller, preconditions, postconditions, and side effects are clear enough at architecture level.

19.2 Minimal Canonical Flow

1. An access surface submits a user, system, module, or agent-originated reasoning need to the kernel. The surface is not authority.
2. **MADREKernel** resolves an explicit valid target module when present; otherwise it falls back to the Core Module role. The kernel does not semantically choose the best module.
3. A **ReasoningModule** receives a **ReasoningRequest** and answers, clarifies, blocks, delegates, creates a plan, or fails inside its own boundary.
4. Structured or delayed work becomes a **ReasoningPlan** containing **ReasoningTask** occurrences.
5. A **Workflow** may be applied by module-owned planning to generate or refine tasks. It is a planning-time blueprint, not an executable action.
6. Execution-time tasks invoke a module-owned **MADREAgent**, **AgentRoutine**, **AgentAction**, or **ModelAction** through a concrete **AgentRequest**.
7. **ModelAction** uses an allowed **ModelBinding**; generated material returns through **AgentResponse** and, when materialized, **ReasoningArtifact**.

8. `KnowledgeRecord`, `KnowledgeCandidate`, and `LearningCandidate` remain separate lifecycle concepts. Produced material does not become knowledge or learning by itself.
9. `PolicyDecision` records boundary outcomes. `RuntimeEvent` records runtime evidence. `RuntimeJournal` appends the local trace.

19.3 Conventions

19.3.1 Type Conventions

- **UUID:** Stable identifier
- **Text:** Human-readable or structured textual description
- **Instant:** Timestamp
- **Ref<T>:** Stable reference to another runtime object
- **List<T>:** Ordered collection
- **Set<T>:** Unordered collection without duplicates
- **Map<K,V>:** Key-value structure
- **Optional<T>:** Absent or present value

19.3.2 Process Conventions

Term	Meaning
<code>minimizeContext</code>	Context adapted to the target scope, with sensitive or unrelated material omitted.
Boundary composition	Construction of valid planning or execution paths from declared module, agent, workflow, routine, action, model, context, storage, privacy, and learning constraints.
Planning-time compatibility	Compatibility used when <code>Workflow</code> creates or refines <code>ReasoningTask</code> objects.
Execution-time compatibility	Compatibility required before <code>AgentRoutine</code> , <code>AgentAction</code> , or <code>ModelAction</code> invocation.
State transition	A guarded process recorded through <code>RuntimeEvent</code> , not a plain enum assignment.

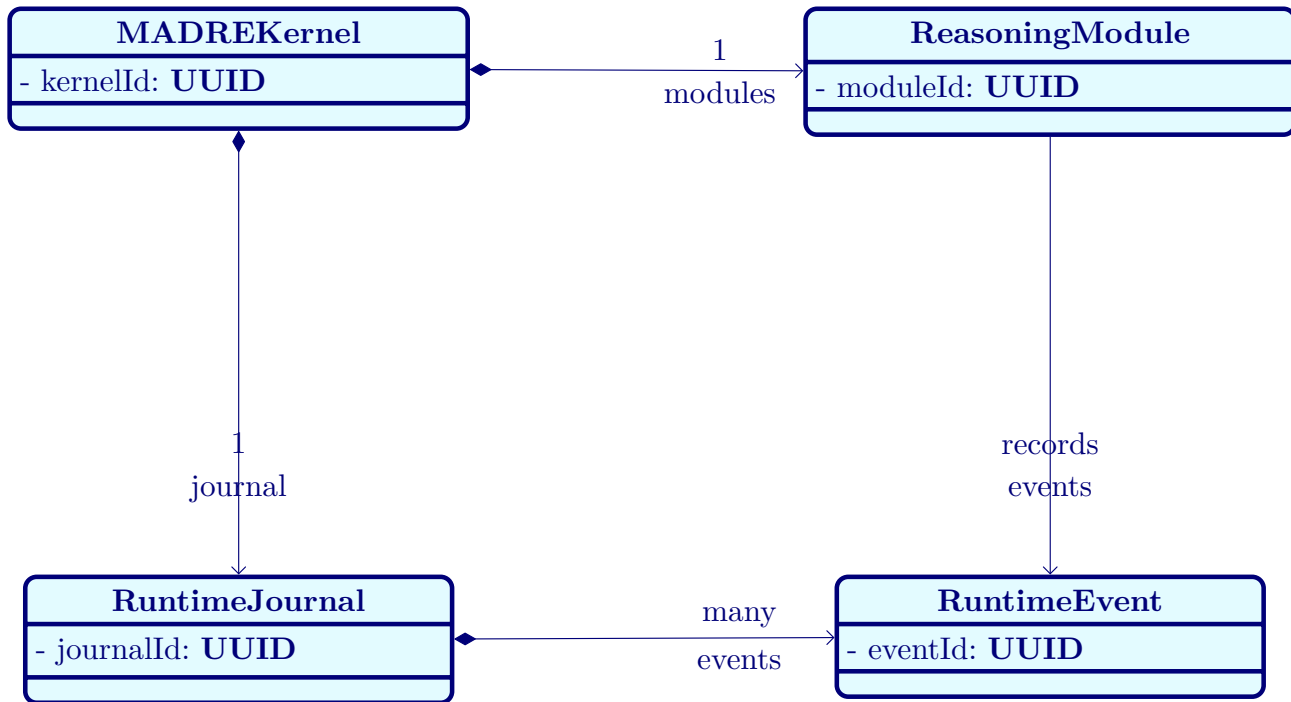
19.3.3 Class Inventory

Accepted classes	Open runtime areas
MADREKernel, ReasoningModule	Scheduler internals, storage ports, access-surface adapters, device-resource bindings.
MADREAgent, SimpleAgent, TwinAgent, ComplexAgent, SystemAgent	Concrete lane scheduling, review implementation, live-interaction adapter methods.
Workflow, AgentRoutine, AgentAction, ModelAction, ModelBinding	Capability metadata taxonomy, model backend adapter details, resource profiles, action subtype catalogues.
ReasoningRequest, ReasoningResponse, ReasoningPlan, ReasoningTask, AgentRequest, AgentResponse	Request status taxonomy, task transition operations, dependency resolution, quality-gate internals.
ContextBundle, ReasoningArtifact, KnowledgeRecord, KnowledgeCandidate, LearningCandidate	Truth taxonomy, lifecycle taxonomy, artifact subtype catalogue, promotion and rollback mechanics.
PolicyDecision, RuntimeEvent, RuntimeJournal,	Event taxonomy, policy-rule engine internals, query/index implementation, evidence payload schema.

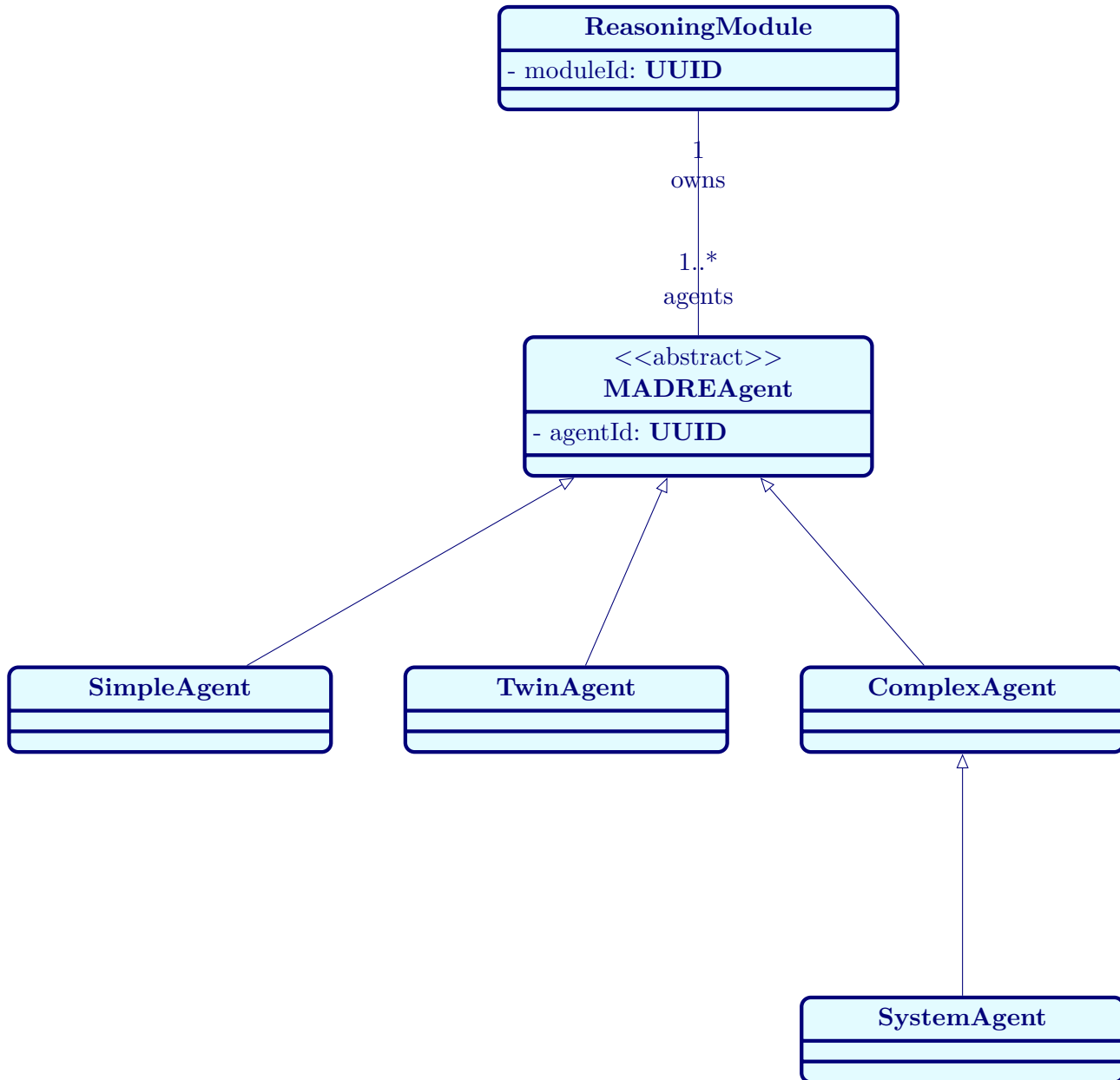
19.4 Accepted UML Registry

19.5 Structural Relations

19.5.1 Runtime Ownership

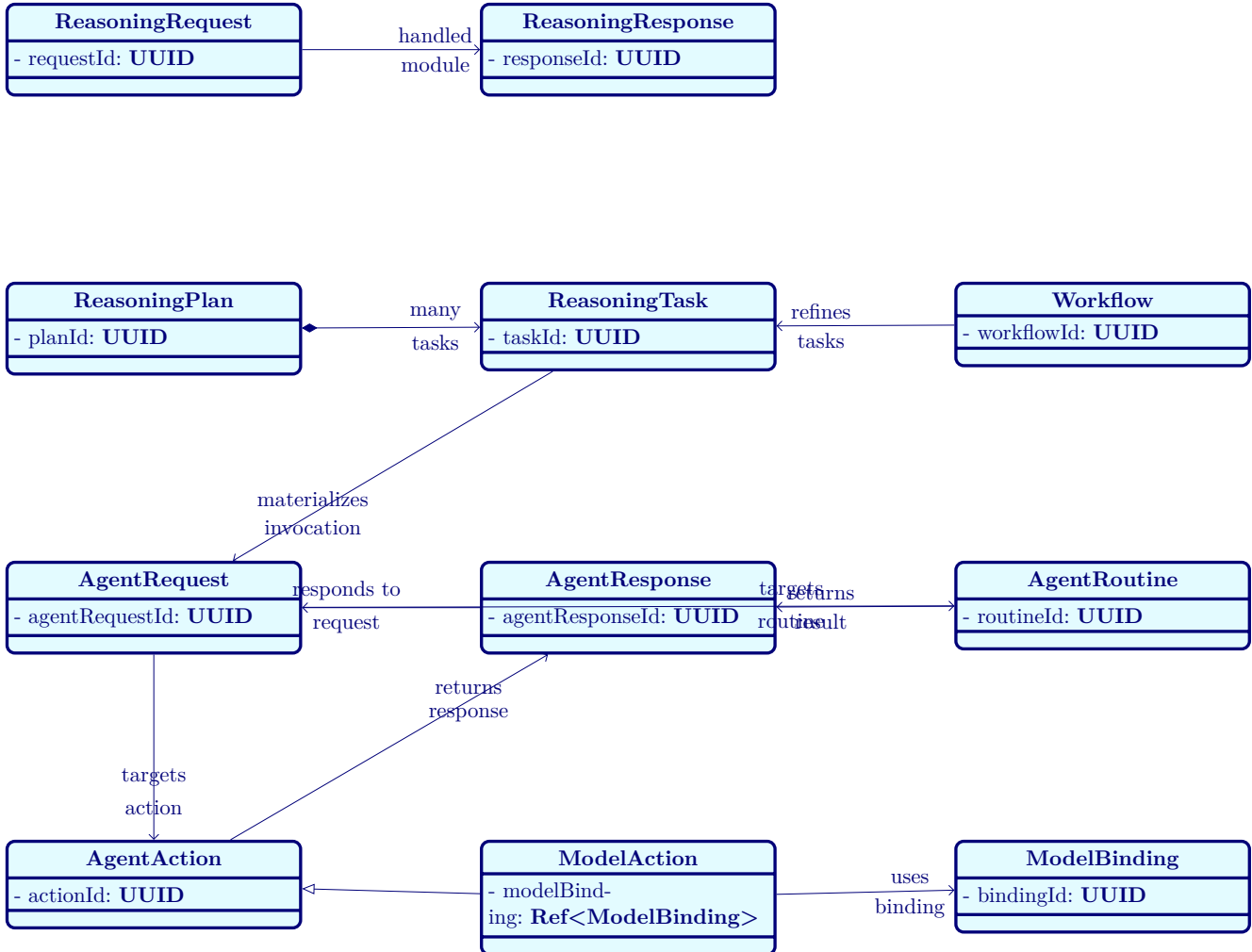


19.5.2 Agent Family

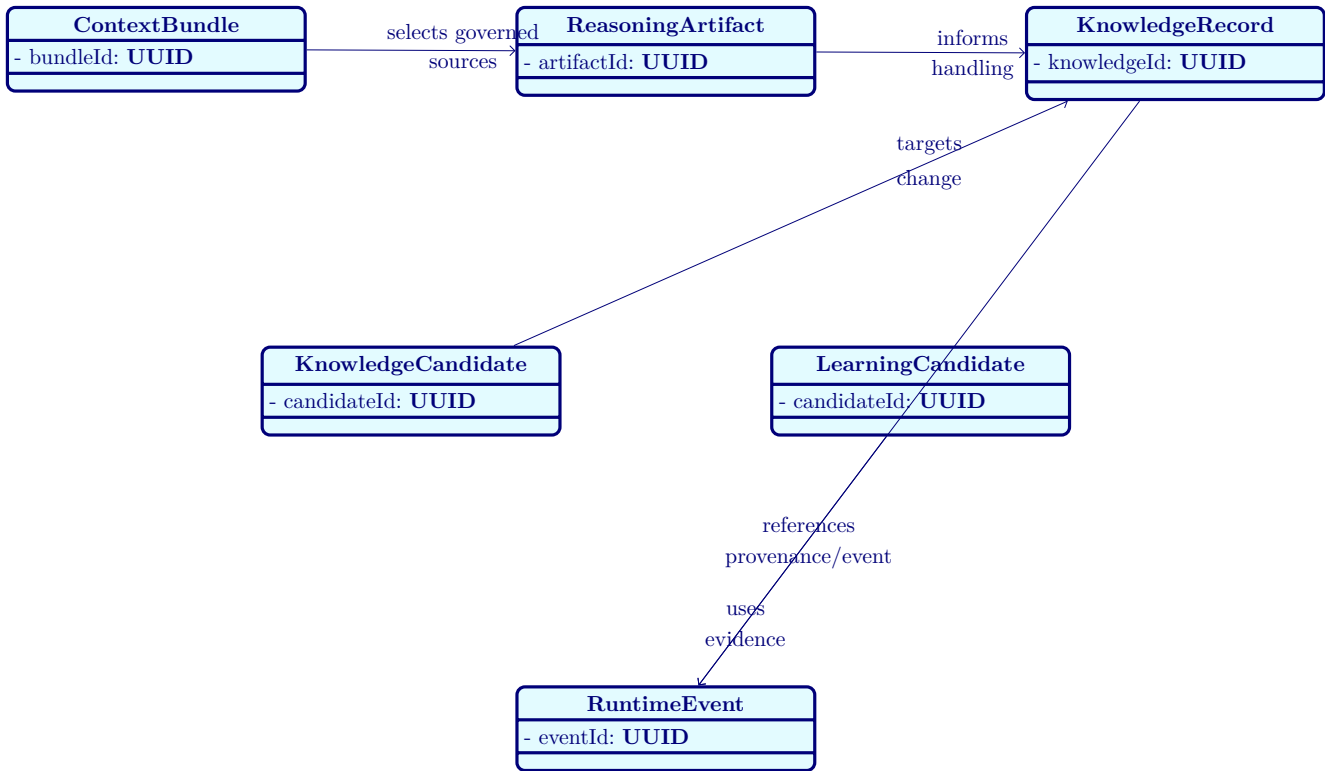


The diagram shows module ownership. Specialization is defined in the UML registry: **SimpleAgent**, **TwinAgent**, and **ComplexAgent** specialize **MADREAgent**; **SystemAgent** specializes **ComplexAgent** and is reserved for module-bound main-agent responsibility.

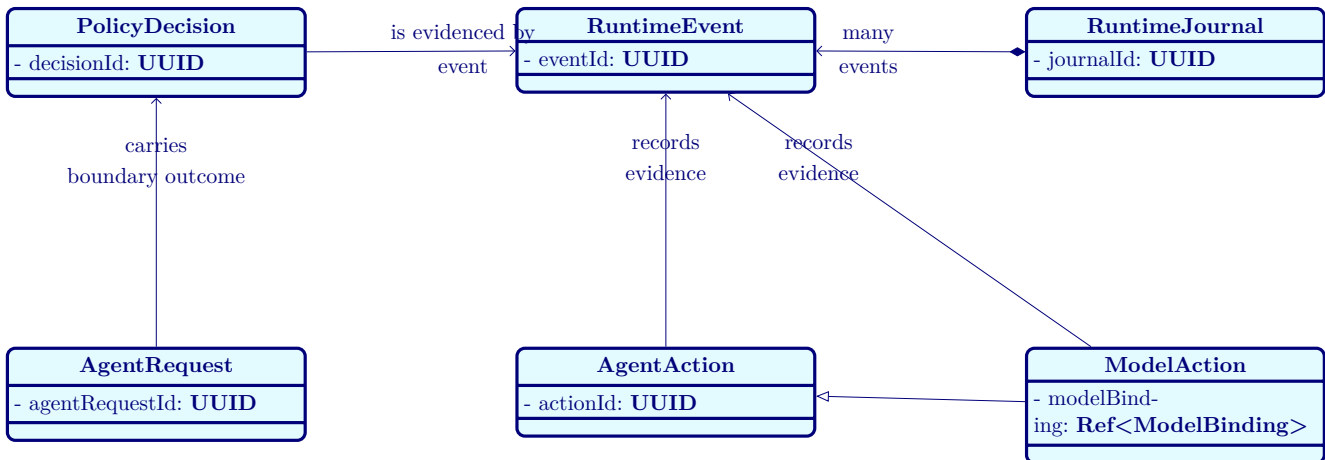
19.5.3 Planning And Invocation



19.5.4 Context, Material, Knowledge, And Learning



19.5.5 Trace And Authority Evidence



19.6 Class Definitions

19.6.1 MADREKernel

MADRE Agentic System

Class Definition
MADREKernel

MADREKernel: MADREKernel coordinates valid targets and runtime trace without becoming the reasoning actor.

- kernelId: UUID

Attributes:

- coreModule: Ref<ReasoningModule>

- moduleRegistry: Set<Ref<ReasoningModule>>

- runtimeJournal: RuntimeJournal

Operations: -NoValue-

Notes:

Deterministic local runtime coordinator.

Registers modules, assigns the Core Module role, resolves explicit module targets, falls back to the Core Module when a target is missing or invalid, schedules delayed work, provides scoped runtime services, and appends events.

It does not reason, semantically route, inspect module-private knowledge, execute actions, invoke models, promote knowledge, or learn.

Storage assignments, scheduler internals, access-surface adapters, and hardware bindings are implementation-defined infrastructure.

19.6.2 ReasoningModule

MADRE Agentic System

Class Definition
ReasoningModule

ReasoningModule: ReasoningModule owns semantic reasoning and local governance inside a module boundary.

Attributes:

- moduleId: UUID
- mainAgent: Ref<SystemAgent>
- agents: Set<Ref<MADREAgent>>
- workflows: Set<Workflow>
- actions: Set<AgentAction>

Operations: -NoValue-

Notes:

Governed reasoning and data-governance unit for a bounded domain, topic, user scope, system scope, or operational scope.

Owns module knowledge, local context governance, allowed agents, workflows, routines, actions, model bindings, learning boundaries, and risk limits.

Handles requests by answering, clarifying, blocking, delegating, creating a plan, or failing.

It does not become the kernel, directly access another module's private data, or widen the authority of its agents and actions.

19.6.3 MADREAgent

MADRE Agentic System

Class Definition
MADREAgent

<AbstractClass>

MADREAgent: MADREAgent executes module-authorized invocations.

Attributes: - agentId: UUID

Operations: -NoValue-

Notes:

Abstract module-owned runtime actor that executes concrete **AgentRequest** objects under module boundaries.

Uses provided context and runtime-constructed execution context, invokes only allowed routines or actions, and returns **AgentResponse** material.

It does not own kernel resources, widen its own authority, or promote produced material into knowledge or learning.

19.6.4 SimpleAgent

MADRE Agentic System

Class Definition
SimpleAgent

SimpleAgent: SimpleAgent directly executes valid requests.

Attributes:

Notes:

Concrete MADREAgent for direct execution where no mandatory review or complex lane behavior is required. It returns one AgentResponse and does not gain reflection, review, or learning authority.

19.6.5 TwinAgent

MADRE Agentic System

Class Definition
TwinAgent

TwinAgent: TwinAgent is justified by the checked-output postcondition.

Attributes:

Notes:

Concrete MADREAgent whose invariant is response production plus mandatory review before final dispatch. The review path does not widen action authority and unchecked material cannot satisfy a task that requires checked output.

19.6.6 ComplexAgent

MADRE Agentic System

Class Definition
ComplexAgent

ComplexAgent: ComplexAgent coordinates deeper behavior without gaining extra authority.

Attributes:

Notes:

Concrete MADREAgent capable of foreground continuity, deeper reasoning supervision, and reflective/evaluative work. Foreground, reasoning, and reflection lanes are behavioral capacities; scheduling and threading are not class attributes. Reflection may propose LearningCandidate objects but cannot activate learning.

19.6.7 SystemAgent

MADRE Agentic System

Class Definition
SystemAgent

SystemAgent: SystemAgent is the module-bound complex agent used for ordinary module operation.

Attributes:

Notes:

Module-bound **ComplexAgent** that serves as the module's main operational agent.
 Its distinguishing property is strong local binding to one **ReasoningModule**, not global authority.
 The Main Agent is a role held by one module-owned **SystemAgent**; it is not a separate class.

19.6.8 Workflow

MADRE Agentic System

Class Definition
Workflow

Workflow: Workflow is reusable module planning structure.

- workflowId: UUID

Attributes: - ownerModule: Ref<ReasoningModule>
 - purpose: Text

Notes:

Module-level planning blueprint that generates or refines **ReasoningTask** occurrences inside a **ReasoningPlan**.
 It is not an actor, not an action, not an agent routine, and not a scheduled object by itself.
 Compatibility is planning-time compatibility under accumulated request constraints.

19.6.9 AgentRoutine

MADRE Agentic System

Class Definition
AgentRoutine

AgentRoutine: AgentRoutine is compound executable agent behavior.

- routineId: UUID

Attributes: - ownerAgent: Ref<MADREAgent>
 - purpose: Text

Notes:

Agent-level compound executable capability.
 It receives **AgentRequest**, may coordinate several lower-level steps or actions, and returns **AgentResponse**.
 It is execution-time capability, not a module workflow and not a replacement for **ReasoningPlan**.

19.6.10 AgentAction

MADRE Agentic System

Class Definition
AgentAction

AgentAction: AgentAction is the smallest bounded executable capability.

- actionId: UUID

Attributes: - ownerModule: Ref<ReasoningModule>
 - purpose: Text

Operations: -NoValue-

Notes:

Smallest executable capability available under module or agent authority.

Executes deterministic, model-backed, integration-backed, or other bounded behavior only through a valid invocation path.

It does not authorize itself, execute from raw user text, or bypass runtime trace.

19.6.11 ModelAction

MADRE Agentic System

Class Definition
ModelAction

ModelAction: ModelAction is model-backed action execution under binding constraints.

Attributes: - modelBinding: Ref<ModelBinding>

Notes:

Specialized **AgentAction** that uses a model binding as part of execution.

It prepares model-backed input from **AgentRequest** and governed context, uses only allowed **ModelBinding**, and returns produced material through ordinary response or artifact paths.

Runtime evidence for model-backed execution is recorded through **RuntimeEvent**; no separate output class is accepted here.

19.6.12 ModelBinding

MADRE Agentic System

Class Definition
ModelBinding

ModelBinding: ModelBinding constrains model-backed execution without becoming model authority.

- bindingId: UUID

- bindingId: UUID

Attributes:

- allowedBackend: Text
- modelUseProfile: Text
- localityMode: Text

Notes:

Configuration boundary that constrains which model-backed execution options a model-backed action or agent may use.

Preserves model substitutability, local-first or authorized-remote limits, reproducibility expectations, and resource discipline.

It is not the model, not a runtime actor, not authority, and does not execute inference.

19.6.13 ReasoningRequest

MADRE Agentic System

Class Definition
ReasoningRequest

ReasoningRequest: ReasoningRequest represents module-level reasoning or delegation need.

- requestId: UUID

- objective: Text

- originRef: Ref<?>

Attributes:

- targetModule: Optional<Ref<ReasoningModule>>
- contextRef: Optional<Ref<ContextBundle>>
- scope: Text
- declaredBoundary: Text

Notes:

Passive request object for a reasoning need. It may originate from a user interaction, system event, module, agent, plan, or delegated task.

Carries objective, origin, explicit target when known, scope, declared boundary summary, and context references.

It does not build plans, schedule itself, execute work, authorize actions, or carry module-private knowledge directly across delegation boundaries.

19.6.14 ReasoningResponse

MADRE Agentic System

ReasoningResponse

ReasoningResponse: ReasoningResponse carries module-level handling output.

- responseId: UUID
- outcome: Text

Attributes:

- materialRef: Optional<Ref<?>>
- artifactRefs: List<Ref<ReasoningArtifact>>
- planRef: Optional<Ref<ReasoningPlan>>

Notes:

Passive module-level response to a ReasoningRequest.

Represents answer, clarification, block, delegation, plan-created, or failure material.

It does not execute work, authorize actions, or promote contained material into knowledge or learning.

19.6.15 ReasoningPlan

MADRE Agentic System

ReasoningPlan

ReasoningPlan: ReasoningPlan owns delayed reasoning task structure.

- planId: UUID
- Attributes:**
- ownerModule: Ref<ReasoningModule>
 - sourceRequest: Ref<ReasoningRequest>
 - tasks: List<Ref<ReasoningTask>>

Notes:

Durable reasoning-continuity object for structured or delayed work.

Owns task occurrences and preserves enough plan structure for scheduling, inspection, dependency management, failure handling, and resumption.

It does not execute itself, allocate kernel resources, bypass module ownership, or promote results into knowledge or learning.

19.6.16 ReasoningTask

MADRE Agentic System

Class Definition
ReasoningTask

ReasoningTask: ReasoningTask records planned work before concrete invocation.

Attributes:

- taskId: UUID
- targetKind: Text
- targetRef: Optional<Ref<?>>
- dependencies: List<Ref<ReasoningTask>>
- requiredContextScope: Text
- requiredBoundaryOutcome: Text

Notes:

Passive planned execution occurrence inside a ReasoningPlan.

Targets a delegated ReasoningRequest, an executable agent capability, an AgentRoutine, an AgentAction, a ModelAction, or a Workflow only when planning expansion or refinement is intended.

It does not contain a prebuilt AgentRequest before execution selection and does not execute itself.

19.6.17 AgentRequest

MADRE Agentic System

Class Definition
AgentRequest

AgentRequest: AgentRequest is the concrete invocation envelope.

Attributes:

- agentRequestId: UUID
- objective: Text
- contextRef: Ref<ContextBundle>
- targetAgent: Optional<Ref<MADREAgent>>
- targetCapability: Optional<Ref<?>>
- effectiveBoundary: Text
- policyDecisionRef: Optional<Ref<PolicyDecision>>

Notes:

Concrete executable invocation context constructed by module orchestration.

Carries only the minimized context and effective boundary needed for one valid agent, routine, action, or model-backed execution.

It does not replace ReasoningRequest, schedule work, or make context valid merely by containing it.

19.6.18 AgentResponse

MADRE Agentic System

Class Definition
AgentResponse

AgentResponse: AgentResponse wraps concrete execution output.

Attributes:

- agentResponseId: UUID
- outcome: Text
- artifactRefs: List<Ref<ReasoningArtifact>>
- eventRefs: List<Ref<RuntimeEvent>>

Notes:

Concrete invocation result returned by a MADREAgent, AgentRoutine, AgentAction, or ModelAction.
 Carries result material, artifact references, block material, refusal material, or failure material.
 It is not knowledge, policy, authority, learning, or routing by itself.

19.6.19 ContextBundle

MADRE Agentic System

Class Definition
ContextBundle

ContextBundle: ContextBundle carries minimized selected context without making it authoritative.

Attributes:

- bundleId: UUID
- ownerModule: Ref<ReasoningModule>
- sourceRefs: List<Ref<?>>
- scope: Text
- sensitivity: Text
- intendedUse: Text

Notes:

Governed context prepared for one request, task, agent invocation, routine invocation, action invocation, or model-backed execution.
 Carries selected material or references while preserving source, scope, sensitivity, purpose, minimization, and revocation semantics.
 Source material does not become instruction or authority by inclusion.

19.6.20 ReasoningArtifact

MADRE Agentic System
Class Definition
ReasoningArtifact

ReasoningArtifact: ReasoningArtifact preserves produced material without granting truth or authority.

Attributes:

- artifactId: UUID
- ownerModule: Ref<ReasoningModule>
- contentRef: Ref<?>
- originEventRef: Ref<RuntimeEvent>
- intendedUse: Text

Notes:

Materialized output or captured result produced by reasoning or action execution.

Represents produced material that may be returned, inspected, exported, referenced, corrected, discarded, or used as input to later handling.

It is not knowledge, truth, learning, policy, or action authority by default.

19.6.21 KnowledgeRecord

MADRE Agentic System
Class Definition
KnowledgeRecord

KnowledgeRecord: KnowledgeRecord is uncertain retained module knowledge.

Attributes:

- knowledgeId: UUID
- ownerModule: Ref<ReasoningModule>
- contentRef: Ref<?>
- provenanceRefs: List<Ref<?>>
- scope: Text
- sensitivity: Text
- intendedUse: Text
- truthLevel: Text
- truthAuthority: Text
- confidence: Text
- lifecycleState: Text

Notes:

Module-owned retained material that may later be used as known context, evidence, preference, style, fiction, procedure, hypothesis, correction, memory, or working belief under explicit metadata.

It is broad uncertain knowledge, not factual truth by default and not a subclass of **ReasoningArtifact** in this baseline.

It does not authorize actions, promote itself across module boundaries, or become learning by being stored.

19.6.22 KnowledgeCandidate

MADRE Agentic System

Class Definition KnowledgeCandidate

KnowledgeCandidate: KnowledgeCandidate quarantines pending knowledge-boundary change.

Attributes:

- candidateId: UUID
- ownerModule: Ref<ReasoningModule>
- proposedChange: Text
- sourceRefs: List<Ref<?>>
- targetKnowledgeRef: Optional<Ref<KnowledgeRecord>>
- status: Text

Notes:

Pending change to a module knowledge boundary.

Represents proposed inclusion, reclassification, correction, ownership transfer, trust-level change, intended-use change, promotion, detachment, invalidation, or related mutation.

It is not active knowledge, not learning, and does not authorize future use until resolved.

19.6.23 LearningCandidate

MADRE Agentic System

Class Definition LearningCandidate

LearningCandidate: LearningCandidate stores proposed behavior improvement without activating it.

Attributes:

- candidateId: UUID
- ownerModule: Ref<ReasoningModule>
- proposedImprovement: Text
- evidenceRefs: List<Ref<?>>
- scope: Text
- rollbackRefs: List<Ref<?>>
- status: Text

Notes:

Evaluated proposed improvement to module behavior, evaluation, routing, workflows, routines, context selection, trust calibration, model selection, or other module function.

Remains inactive until promoted through the applicable learning boundary.

Saving KnowledgeRecord material is not learning by itself.

19.6.24 PolicyDecision

MADRE Agentic System

Class Definition PolicyDecision

PolicyDecision: PolicyDecision records a boundary outcome without granting authority by itself.

Attributes:

- decisionId: UUID
- outcome: Text
- reason: Text
- affectedRef: Optional<Ref<?>>
- eventRef: Optional<Ref<RuntimeEvent>>

Notes:

Passive record of an explicit boundary or authorization outcome.

Records allow, block, or authorization-required outcomes and links them to affected action, data, autonomy, or boundary evidence when known.

It does not execute policy logic, authorize by itself, or override the composed boundary mechanism.

19.6.25 RuntimeJournal

MADRE Agentic System

Class Definition RuntimeJournal

RuntimeJournal: RuntimeJournal preserves append-only runtime history.

Attributes: - journalId: UUID

Operations: -NoValue-

Notes:

Append-only local runtime trace mechanism.

Appends **RuntimeEvent** entries and stores or references payloads needed to inspect request handling, plan lifecycle, task execution, context construction, model-backed action execution, action execution, produced material, policy outcomes, failures, candidates, correction, rollback, deletion, and recovery.

It is not a reasoning actor, knowledge repository, general logging framework, or authorization mechanism.

19.6.26 RuntimeEvent

MADRE Agentic System

Class Definition
RuntimeEvent

RuntimeEvent: RuntimeEvent records runtime evidence without authorizing it.

- eventId: UUID
- occurredAt: Instant
- eventType: Text
- Attributes:**
 - actorRef: Optional<Ref<?>>
 - affectedRef: Optional<Ref<?>>
 - summary: Text
 - payloadRef: Optional<Ref<?>>

Notes:

Append-only record of something relevant that happened in the runtime.

Records request receipt, target resolution, plan creation, scheduling, task transition, context construction, boundary outcome, action execution, model-backed execution, artifact creation, response production, failure, correction, rollback, invalidation, supersession, deletion, detachment, knowledge-candidate transition, or learning-candidate transition.

It does not authorize what happened, execute behavior, make material true, or replace domain objects.

19.7 Validation Checklist

1. The accepted class inventory contains exactly the 26 baseline runtime classes.
2. The kernel remains deterministic coordination and target resolution, not semantic reasoning.
3. Core Module and Main Agent remain roles, not separate classes.
4. Modules own reasoning, knowledge, agents, workflows, routines, actions, and local governance.
5. Workflow is planning-time task generation or refinement, not executable work.
6. AgentRoutine, AgentAction, and ModelAction are execution-time capabilities invoked through concrete AgentRequest.
7. ReasoningTask does not contain a prebuilt AgentRequest before execution selection.
8. Model-backed execution is ModelAction plus ModelBinding, ordinary produced material, and RuntimeEvent evidence.
9. Produced material does not become knowledge, truth, policy, action authority, routing, behavior, or learning by itself.
10. KnowledgeRecord, KnowledgeCandidate, and LearningCandidate preserve separate lifecycle semantics.

11. Boundary enforcement is not reduced to a loose policy flag or generic container.
12. Storage mechanisms remain implementation-defined infrastructure until a stable domain boundary is proven.

Glossary

Term	Definition
AgentAction	Smallest executable capability available under module or agent authority.
AgentRequest	Concrete invocation envelope constructed for an agent, routine, action, or model-backed action.
AgentResponse	Concrete result envelope returned by an agent, routine, or action.
AgentRoutine	Agent-owned compound executable capability that receives AgentRequest and returns AgentResponse .
Boundary composition	Runtime construction of a valid path from accumulated module, agent, workflow, routine, action, model-use, context, storage, privacy, and learning constraints.
Core Module	Kernel-assigned role held by one compatible ReasoningModule for ordinary interaction fallback; it is not a module subtype.
ComplexAgent	MADREAgent subtype capable of foreground continuity, deeper reasoning supervision, and reflection without gaining extra authority.
ContextBundle	Minimized selected context for a specific runtime purpose; inclusion does not make source material instruction or authority.
KnowledgeCandidate	Pending change to a module knowledge boundary, not active knowledge and not learning.
KnowledgeRecord	Broad uncertain retained module knowledge with provenance, scope, sensitivity, intended use, truth authority, confidence, correction, and lifecycle metadata.

Term	Definition
LearningCandidate	Evaluated proposed module improvement that remains inactive until promoted through the learning boundary.
MADREKernel	Deterministic local coordinator for module registration, target resolution, Core Module assignment, scheduling, runtime services, and event append.
MADREAgent	Abstract module-owned runtime actor that executes concrete invocations under module boundaries.
Main Agent	Role held by one module-owned SystemAgent ; it is not a separate class and grants no cross-module authority.
ModelAction	AgentAction specialization that uses an allowed ModelBinding and returns ordinary produced material with RuntimeEvent evidence.
ModelBinding	Configuration boundary constraining model-backed execution options without becoming the model or authority.
PolicyDecision	Passive record of an allow, block, or authorization-required boundary outcome.
ReasoningArtifact	Produced or materialized output; not knowledge, truth, policy, authority, or learning by default.
ReasoningModule	Governed reasoning and data-governance unit that owns module knowledge, agents, workflows, routines, actions, context governance, and local boundaries.
ReasoningPlan	Durable reasoning-continuity object containing planned task occurrences.
ReasoningRequest	Passive module-level reasoning or delegation need.

Term	Definition
ReasoningResponse	Passive module-level response representing answer, clarification, block, delegation, plan-created, or failure material.
ReasoningTask	Planned execution occurrence inside a ReasoningPlan ; concrete AgentRequest is built later at execution selection.
RuntimeEvent	Append-only runtime fact or evidence record.
RuntimeJournal	Append-only local trace mechanism for runtime events and inspection payload references.
SimpleAgent	Direct-execution MADREAgent subtype.
SystemAgent	Module-bound ComplexAgent used for ordinary module operation and Main Agent role assignment.
TwinAgent	MADREAgent subtype justified by mandatory checked-output behavior.
Workflow	Module-owned planning blueprint that creates or refines ReasoningTask occurrences; not executable work.
Read-first console	Local inspection surface that exposes status and trace without becoming a privileged mutation path.
Recovery path	Defined correction, rollback, deletion, compensation, blocking, invalidation, supersession, or detachment behavior for an outcome or state transition.